Graph Prompt Learning: A Comprehensive Survey and Beyond

Abstract—Artificial General Intelligence (AGI) has transformed numerous fields, yet graph data remains comparatively less explored than text and vision in foundation-model-centered workflows. This survey reviews graph prompt learning from the perspective of cross-modality, cross-domain, and cross-task generalization, and organizes the area with a unified framework covering prompt tokens, token structures, and insertion patterns. We revisit the relationship between graph pre-training and prompting, discuss how prompts align downstream tasks with pretext objectives, and summarize representative prompt tuning strategies. Our taxonomy organizes over one hundred representative works, including recent studies through early 2026, across pre-training strategies, prompt design, multi-modal prompting, and graph domain adaptation. We also discuss ProG, a Python library for graph prompting, together with a companion repository for collecting relevant papers, benchmarks, and code resources. Finally, we outline current challenges and future directions to support continued research on graph prompting for graph representation learning and related AGI-oriented applications. The companion website and ProG can be accessed by https://github.com/WxxShirley/Awesome-Graph-Prompt and https://github.com/sheldonresearch/ProG, respectively.

Index Terms—graph prompt, graph pre-training, graph learning, artificial general intelligence.

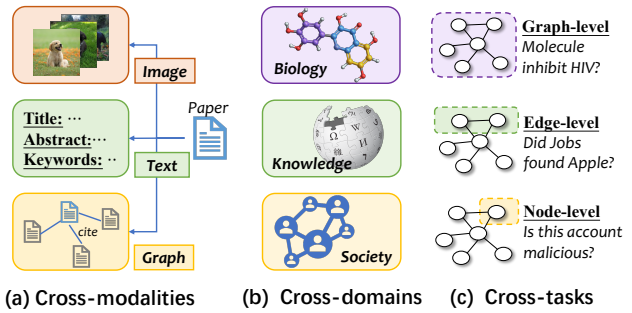


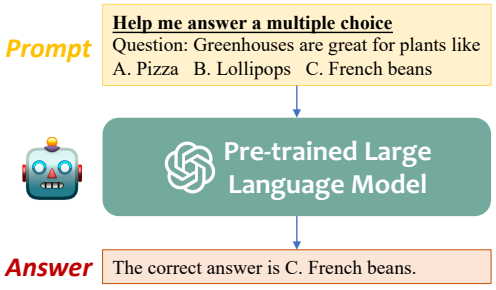
Fig. 1: The Most Frequent Problems towards Artificial General Intelligence. (a) Cross-modalities, (b) Cross-domains, (c) Cross-tasks.

1 Introduction

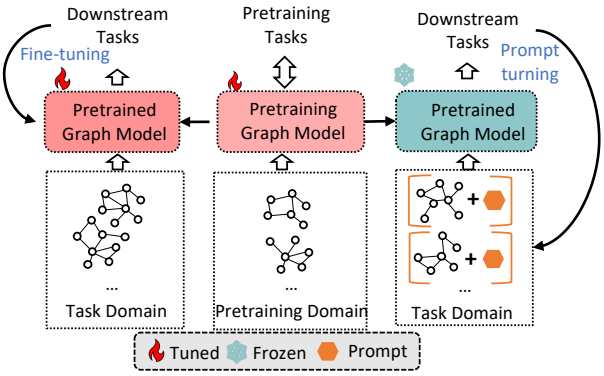
In an era marked by the rapid evolution of Artificial General Intelligence (AGI), there emerged many fantastic applications with AGI techniques such as ChatGPT in Natural Language Processing (NLP) and Midjourney in Computer Vision (CV). AGI has greatly improved our lives, making our work more efficient and freeing us from repetitive tasks to focus on more creative endeavors. However, when it comes to working with graph data, AGI applications are still in their early stages compared with the huge success in NLP [10, 2, 52] and CV areas [95, 118]. In our increasingly interconnected world, understanding and extracting valuable insights from graphs is crucial. This places AGI applied to graph data at the forefront of both academic and industrial interest [51, 124, 112], with the potential to redefine fields like drug design [72, 68] and battery development [94], etc.

However, realizing this vision is never easy. Figure 1 illustrates this landscape for recent research in Artificial General Intelligence, from which we can see that there are at least three fundamental problems in technique: ***(1) How to make the model general for different modalities?*** Within the NLP and CV areas, there have been many commercial models that can understand and translate information across these modalities [10, 118, 2]. For example, models like BERT [10] and GPT-3 [2] have demonstrated the ability to perform tasks that involve both textual and visual information. However, in the context of graph data, the harmonization of information from multiple modalities remains largely uncharted territory [47]. ***(2) How to make the model general for different domains?*** For the cross-domain issue, transfer learning has proven effective, enabling models to apply knowledge learned from images and text in one domain to another. However, transferring knowledge between different graph domains is very tough because the semantic spaces are not aligned [130] and the structural patterns are also not similar [127], making graph domain adaptation remains a very frontier and not well-solved AGI problem. ***(3) How to make the model general for different tasks?*** Currently, most graph research on transfer learning focuses on the third problem, how to leverage the pre-trained graph knowledge in the same graph domain to perform different graph tasks (like node classification, link prediction, graph classification, etc) [82, 55, 84, 13, 129, 34, 78, 20]. However, compared with the huge success in NLP and CV, task transferring within the same graph domain is still primitive with far fewer instances of successful industrial applications. While the AGI research boasts notable achievements in many linear data like images, text [71, 10, 2], and videos [95, 118], the fundamental problems within the realm of graph data remain underexplored.

These concerns have led to a growing consensus within the AI community: exploring a more efficient approach to extract the preserved knowledge from large models instead of tuning these large models repetitively[18, 43]. At the core of recent AGI technology, prompt learning has presented huge potential to solve the above problems and demonstrated remarkable success in NLP and CV applications [69, 90, 52]. Prompt learning is the art of



(a) language prompt



(b) graph prompt v.s. graph fine-tune

Fig. 2: Language prompt v.s. Graph prompt. A textual prompt is added to the original input to let the pre-trained large language model perform a new task (e.g. the multi-choice question-answering). Inspired by the language prompt, a graph prompt can be also used with graphs in the same way.

designing informative prompts to manipulate input data for the pre-trained foundation models. Figure 2 shows an example of a textual-format prompt applied to a pre-trained language model to directly perform downstream inference tasks. By reformulating downstream tasks into pre-training tasks, this approach avoids the need for extensive model tuning and efficiently extracts preserved knowledge [2, 38]. Since its powerful capabilities in data manipulation, task reformulation, and extraction of significant insights, prompting is very promising to address the aforementioned cross-modalities, cross-domains, and cross-task challenges in one way. Compared with large models, the prompt is usually very lightweight and can efficiently extract useful knowledge by reducing the extensive computational resources caused by repeat tuning of these large models [43, 77].

Intuitively, linear structural data like text and images can be perceived as specific instances of a more general non-linear data structure like graphs (a.k.a networks). For instance, a sentence can be treated as a graph path, with words as nodes, and an image can be viewed as a grid graph, where each pixel serves as a graph node. This insight encourages some researchers to explore the transference of successful prompting techniques from existing linear data to the graph area for similar concerns [82, 55, 84, 13, 129, 59, 23, 4, 21].

However, some further studies have found that the graph prompt is very different from their counterparts in the NLP area [84]. **First**, the design of graph prompts proves to be a far more intricate endeavor compared to the formulation of language prompts. Classic language prompts often comprise predefined phrases or learnable vectors appended to input text [2, 18]. Here, the primary focus lies in the content of the language prompt. However, we actually do not know what a graph prompt looks like. A graph prompt not only contains the prompt "content" but also includes the undefined task of determining how to structure these prompt tokens and seamlessly integrate them into the original graph. **Second**, the harmonization of downstream graph problems with the pre-training task is more difficult than language tasks [55, 84]. For example, a typical pre-training approach for a language model is to predict a masked word by the model [10]. Then many downstream tasks such as question answering, and sentiment classification can be easily reformulated as word-level tasks [52]. Unlike NLP, where pre-training tasks often share a substantial task sub-space, graph tasks span node-level [22], edge-level [121], and graph-level objectives [80, 83], making pre-training pretexts less adaptable. **Third**, compared with prompts in NLP which are usually some understandable phrases, graph prompts are usually less intuitive to non-specialists. The fundamental nature and role that graph prompts play within the graph model remain somewhat elusive without comprehensive analysis. In light of these intricacies questions, there is a pressing need for delving deeper into the potential of graph prompts in AGI, thereby paving the way for a more profound understanding of this evolving frontier within the broader data science landscape.

The literature on graph prompting has expanded rapidly in the past few years, but it is still fragmented across different task settings, graph modalities, and adaptation strategies. Existing works often emphasize different perspectives, making direct comparison difficult and slowing the systematic accumulation of knowledge in this area. This survey therefore provides a structured review of how graph prompting addresses the three foundation problems towards AGI discussed above. Beyond that, we further organize the area through the following detailed research questions (RQs):

- **RQ1: How to understand existing work with a unified framework since they are very different?** Our main focus is to understand the various methods used in the field of graph prompting. We want to bring together all these different approaches and ideas to create a single, cohesive framework. This framework will help us thoroughly grasp the existing research and provide a strong foundation for future research in this area.
- **RQ2: Why Prompt? What's the Nature of Graph Prompt?** In this part of our study, we aim to understand why prompts are important. We want to uncover the fundamental aspects of graph prompts. What exactly do graph prompts do in graph problems? How do they fit into the complex graphs, and how do they help us achieve the broader goal of creating AI systems that can handle graph data effectively? These questions highlight the significant role that graph prompts play in shaping the future of AI when dealing with graph information.
- **RQ3: How to Design Graph Prompts?** Designing good graph prompts is a complex task. In this part, we explore the technical details of designing graph prompts: what do they look like, how do they align

downstream tasks and the pre-train task, and how they are learned? These important questions focus on the skill of making prompts that work well with the complexities of graph data, helping researchers make better prompts.

- **RQ4: How to Deploy Graph Prompts in Real-world Applications?** Easy-to-use toolkits for graph prompting are still limited, and many potential applications remain under-explored. This research question focuses on making graph prompts practical for use in real-world situations with an easily extensible programming package.
- **RQ5: What Are the Current Challenges and Future Directions in Graph Prompting?** This question guides us to look at the challenges we're dealing with today and the way forward. By tackling these important questions, we hope to provide a roadmap for ongoing graph-prompting research.

To answer the first research question (RQ1), we propose a unified framework to analyze graph prompt learning work. Our framework casts the concept of a graph prompt into prompt tokens, token structures, and inserting patterns. This higher-level perspective offers clarity and comprehensiveness, providing readers with a structured understanding of the field and a stable lens for comparing later variants.

To answer the second research question (RQ2), we explore the correlations between prompts and existing graph models through the lenses of flexibility and expressiveness and then present a perspective to uncover the nature of graph prompts. Unlike most prompt learning surveys in NLP areas [52] that treat prompting mainly as a way to fill the gap between pre-training tasks and downstream tasks, we show that graph prompts and graph models are interconnected on a deeper level. This perspective helps clarify why prompt learning holds promise in the graph area and what distinguishes it from traditional fine-tuning methods [33].

To answer the third research question (RQ3), we introduce a comprehensive taxonomy that covers over one hundred representative works, including recent additions through early 2026. Our taxonomy dissects these works, categorizing them according to node-level, edge-level, and graph-level tasks, thereby aligning them with the broader context of the pre-training task. This helps readers form a clearer understanding of prompt mechanisms within the whole "pre-training and prompting" workflow.

To answer the fourth research question (RQ4), we developed ProG (prompt graph) [1], a unified Python library to support graph prompting. Additionally, we maintain a companion repository[2] that collects graph prompt papers, benchmark datasets, and available code implementations. By providing this accessible ecosystem, we aim to support researchers and practitioners working on graph prompting more effectively.

Beyond these, our survey goes a step further with an introduction of potential applications, a thoughtful analysis of the current challenges, and a discussion of future directions, thus providing a comprehensive roadmap for the evolution of this vibrant and evolving field (RQ5). Our contributions are summarised as follows:

- **Enabling Comprehensive Analysis.** We propose a unified framework for analyzing graph prompt learning work, providing a comprehensive view of prompt tokens, token structures, and inserting patterns.
- **Novel Perspectives on Prompt-Model Interplay.** We offer a deeper discussion of the nature of graph prompts. Unlike traditional work that treats prompts mainly as a way to fill the gap between downstream tasks and the pre-train task, we explore the flexibility and expressiveness issues of graph models and develop a more thorough perspective on the interplay between prompts and existing graph models.
- **A Systematic Taxonomy of Graph Prompting.** We systematically revisit over one hundred representative works in the domain of graph prompting, including recent additions through early 2026. This taxonomy not only organizes these contributions but also furnishes readers with a comprehensive understanding of prompt mechanisms within the overarching "pre-training and prompting" workflow.
- **Empowering the Graph Prompting Ecosystem.** We developed ProG, a Python library supporting graph prompting, and a companion repository for collecting graph prompt research resources.
- **Charting a Path Forward.** A detailed exploration of current challenges and future directions in the field.

Roadmap. The rest of this survey is organized as follows: we present our survey methodology in section 2, followed by preliminaries in section 3, the introduction of pre-training methods in section 4, prompting methods for graph models in section 5. We discuss potential applications of graph prompt in section 7 and present our developed library ProG in section 8. In section 9, we summarize our survey with current challenges and future directions. Section 10 concludes the survey and presents the contribution declaration of the authors.

2 Survey Methodology

2.1 Research Objectives

This survey introduces graph prompting from the broader perspective of artificial general intelligence (AGI). We first present three fundamental problems towards AGI in Table 1. Prompt learning has been demonstrated as a promising solution to these problems in many linear data such as text [2, 18], images [95], etc. However, whether the prompt technique can still solve these problems in the graph area, is not clearly discussed. Through this survey, we wish to figure out how the graph prompt potentially helps graph models to be more general across various tasks and domains, and how it generalizes the foundation models to interact with other modalities (e.g. text, image, etc). Beyond the above common problems of AGI in NLP, CV, and graph areas, graph prompting is usually very different from its counterparts in NLP and CV areas, leading to many detailed questions as shown in Table 1.

1. https://github.com/sheldonresearch/ProG
2. https://github.com/WxxShirley/Awesome-Graph-Prompt

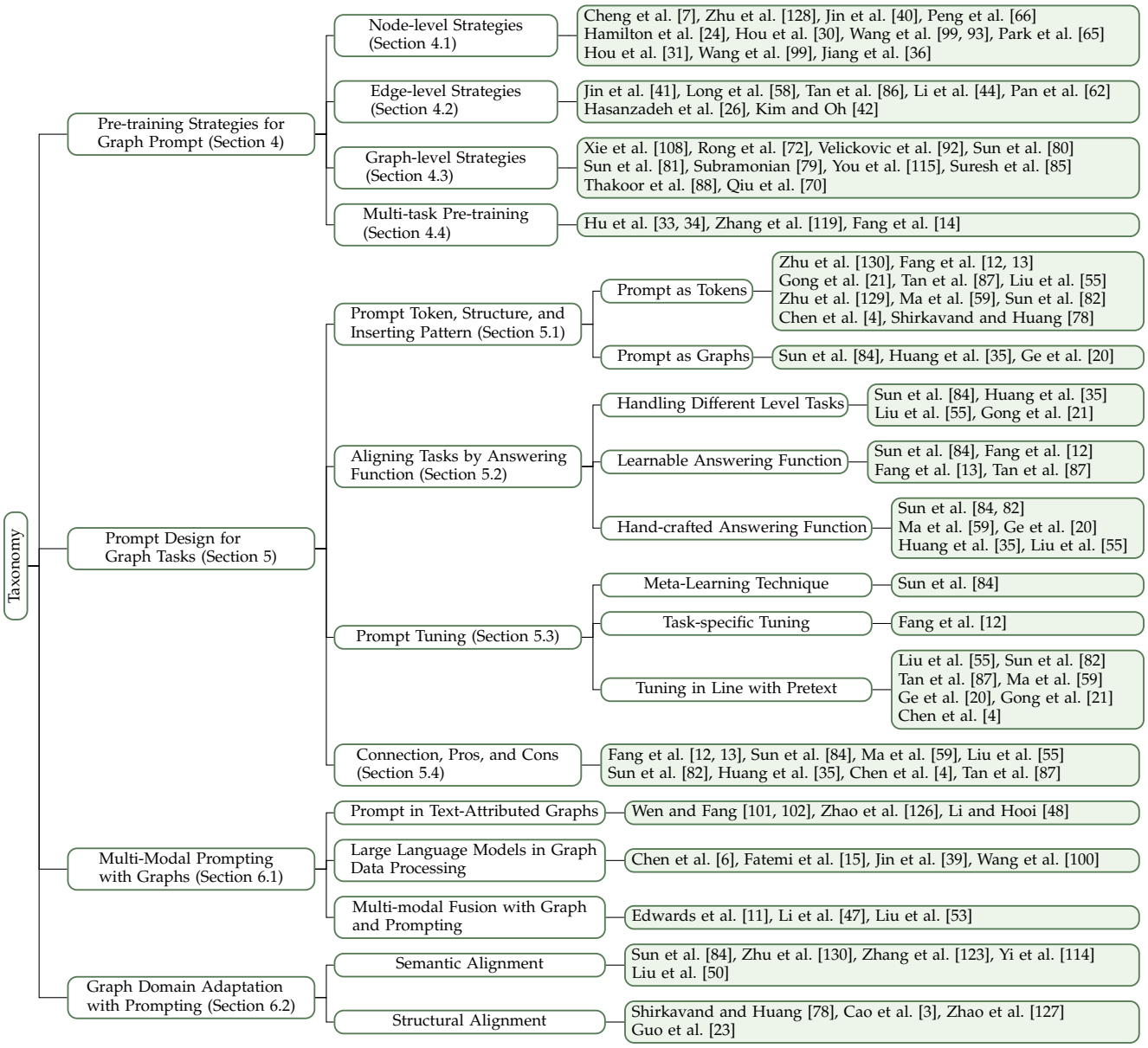


Fig. 3: Taxonomy of this Survey with Representative Works.

2.2 Taxonomy

The taxonomy of this survey is presented in Figure 3, which is intricately designed to categorize graph prompts based on their specific applications and functionalities, providing a structured approach to understanding their role in AGI.

(1) Pre-training Strategies for Graph Prompt.

Since prompting techniques mostly seek to reformulate downstream tasks to the pre-training tasks, they are highly customized for detailed pre-training approaches, thus we briefly discuss the representative pre-training work in the graph area before we formally introduce graph prompt content. We split existing pre-training approaches into node-level, edge-level, graph-level, and multi-task pre-training strategies. Next, we present how different prompting ideas reformulate various downstream tasks to the corresponding pre-training tasks in the graph area. By revisiting existing pre-training literature, we will be more clear about the role of graph prompts in the whole "pre-training and prompting" framework.

(2) Prompting Methods in Graph Areas.

Aiming at the three foundation problems mentioned in the research objectives (P1-P3 in Table 1), we analyze graph prompting from three aspects: i. prompt design for graph tasks (Section 5); ii. multi-modal prompting with graphs (Section 6); and iii. graph domain adaptation with prompting techniques (Section 6.2). Within each aspect, we present a detailed discussion related to the five specific problems in the graph prompt area (Q1-Q5 in Table 1).

i. Prompt Design for Graph Tasks. In this section, we propose a unified framework to analyze existing works on graph prompt design. Our framework treats existing graph prompts with three key components: *prompt tokens*, which preserve prompt content as vectors; *token structures*, which indicate how multiple tokens are organized; and *inserting patterns*, which define how to combine graph prompt with the original graphs. Beyond that, we also carefully analyze

TABLE 1: Research Objectives

Foundamental Problems towards AGI	P1: How to Make the Model General for Different Modalities? (Section 6.1)
	P2: How to Make the Model General for Different Domains? (Section 6.2)
	P3: How to Make the Model General for Different Tasks? (Section 5)
Detailed Questions of Graph Prompt	Q1: How to Understand Existing Work with a Unified Framework? (Section 5.1)
	Q2: What's the Nature of Graph Prompt? (Section 3.5, Section 5.4)
	Q3: How to Design Graph Prompts? (Section 5)
	Q4: How to Deploy Graph Prompts in Real-world Applications? (Section 7, Section 8)
	Q5: What Are the Current Challenges and Future Directions? (Section 9.1)

how these works design the prompt answering function, which means how they get results for the downstream tasks from their prompts. We also summarize three representative methods to learn appropriate prompts, including meta-learning techniques, task-specific tuning, and tuning in line with pretext. In the end, we further discuss these works in Section 5.4 to see their intrinsic connections with pros and cons.

ii. Multi-modal Prompting with Graphs. In this section, we briefly present how graph prompts work in a text-attributed graph, which can be seen as the fusion of text and graph modalities. With the progress of large language models (LLMs), the fusion of text and graph data has become easier and has aroused a lot of work on this topic. Since the topic of integrating LLMs with graphs has been well summarized in [49], we won't present too much in this survey. Instead, we only briefly discuss some representative works in this area that focus on the prompt area.

iii. Graph Domain Adaptation through Prompting Techniques. In this section, we introduce related work from two branches. The first branch presents works solving semantic alignment across different graph domains, and the second branch presents structural alignment.

2.3 Literature Overview

In this survey, we reviewed over one hundred representative papers, with particular attention to conference, journal, and arXiv works released through early 2026. These references cover venues including but not limited to NeurIPS, SIGKDD, The Web Conference, ICLR, CIKM, ICML, IJCAI, EMNLP, SIGIR, ACL, AAAI, WSDM, and TKDE. Most of these venues are ranked as CCF A[3] or CORE A*[4]. We also include recent arXiv papers to reflect fast-moving developments that have not yet fully stabilized into journal or conference versions. Figure 4a presents the venue distribution of the currently collected corpus, and Figure 4b summarizes the top 15 keywords appearing in paper titles. These statistics provide a high-level snapshot of the literature covered in this survey.

3. https://www.ccf.org.cn/Academic_Evaluation/By_category
4. https://www.core.edu.au/conference-portal

2.4 Connection to Existing Work

The most directly related surveys to the present manuscript are the graph-prompt-centered reviews by Wu et al. [106], Long et al. [57], and Fu et al. [17]. The survey of Wu et al. [106] captured the first wave of graph prompting studies and mainly summarized techniques, applications, and challenges when the method space was still relatively small. The later survey of Long et al. [57] expanded the discussion toward design principles, applications, benchmark issues, and open problems, with particular emphasis on prompt tokenization, task alignment, and prompt tuning strategies. More recently, Fu et al. [17] organized the area around the "pre-train and prompt" workflow and classified methods from data-level, representation-level, and task-level perspectives, giving a more mechanism-oriented view of graph prompting.

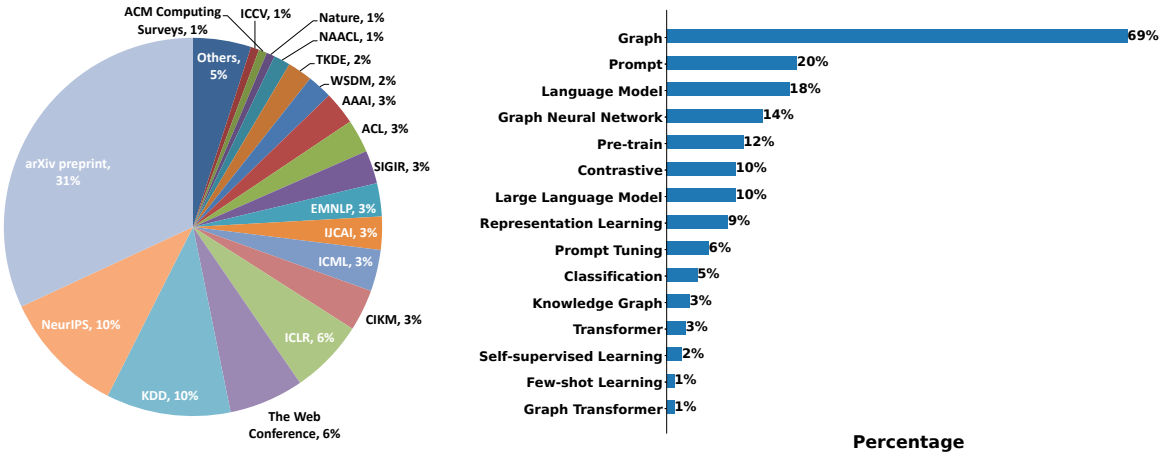
Compared with these surveys, the present manuscript keeps a stronger focus on a unified decomposition of graph prompts into prompt tokens, token structures, insertion patterns, answering functions, and prompt tuning strategies, while also explicitly connecting graph prompting to pre-training strategies, multi-modal prompting, and graph domain adaptation within one taxonomy. In addition, this revised version updates the coverage through early 2026 and preserves an engineering-oriented perspective through the discussion of applications and tool support such as ProG. Broader surveys on graph foundation models, graph-LLM integration, or graph self-supervised learning [51, 49, 109, 117] remain useful background references, but they are not the primary comparison targets here because graph prompting is only one part of their scope.

3 Preliminaries

Graph representation learning has been a topic of extensive research over the past few decades. This journey, illustrated in Figure 5, has witnessed the evolution from shallow embedding methods to supervised graph neural networks, transitioning from the fine-tuning paradigm to the emerging prompting paradigm. In this section, we will provide an overview of the fundamental notations employed in this survey, delve into the historical developments of graph representation learning, explore the pre-training and fine-tuning paradigm, and trace the evolution of prompt-based learning. Most importantly, we will present a novel perspective focusing on flexibility and expressiveness, shedding light on why prompts offer a promising solution to address the limitations of existing graph representation learning methods.

3.1 Notations

Let a graph instance denoted as $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$, where $\mathcal{V} = \{v_1, v_2, \ldots, v_N\}$ represents the node set containing N nodes. The edge set $\mathcal{E} \in \mathcal{V} \times \mathcal{V}$ describes the connection between nodes. Each node v_i is associated with a feature vector represented as $\mathbf{x}_i \in \mathbb{R}^D$. To characterize the connectivity within the graph, we employ the adjacency matrix denoted as $\mathbf{A} \in \{0,1\}^{N \times N}$, where the entry $\mathbf{A}_{ij} = 1$ if and only if the edge $(v_i, v_j) \in \mathcal{E}$.



(a) Venue distribution of collected papers.

(b) Top 15 Keywords appeared in titles of collected papers

Fig. 4: Statistics of the collected papers

3.2 Graph Representation Learning

The last decades have witnessed a notable surge in the development of graph representation learning techniques. These approaches can be broadly categorized into two main branches: shallow embedding methods and deep graph neural networks (GNNs). The shallow embedding approach is centered on mapping nodes into lower-dimensional, learnable embeddings, enhancing their applicability in various downstream tasks, as exemplified by node2vec [22] and DeepWalk [67]. On the other hand, the deep GNNs maintain the input node features as constants and optimize deep graph model parameters for specific tasks, leading to more expressive representation capabilities, as seen in methods such as Graph Convolution Networks (GCN) [60] and GraphSAGE [24].

Shallow embedding approaches make input node features learnable parameters, aiming at encoding nodes in a manner that retains the original network's similarity structure. According to node similarity definition, these methods can be categorized as factorization-based [61, 120] and random walk approaches [22, 67, 98, 97]. Despite the flexibility that shallow embedding methods offer for various downstream tasks, they are constrained by their inability to generate embeddings for nodes not encountered during training. Additionally, these approaches lack the capability to incorporate node features. Therefore, more "deeper" methods, specifically those based on graph neural networks, have been developed to address these limitations.

Most deep GNNs follow a message-passing schema and use a more complex encoder, resulting in powerful expressiveness in graph representation. The representative neural network structure is convolutional graph neural networks (ConvGNNs), which comprise spectral [9, 29] and spatial methods [60, 24, 91, 76, 111]. While these methods have exhibited remarkable capabilities in various graph-based applications, their reliance on task-specific supervision imposes constraints on their adaptability and generalizability, particularly when dealing with tasks that have limited labeled data.

In summary, shallow embedding methods offer flexibility, preserving network structure and node content for straightforward graph analytic tasks. However, they lack expressiveness and the ability to encapsulate additional node features. Conversely, GNNs provide more expressive graph representations but require task-specific training data, limiting their transferability. Hence, it calls for a graph learning mechanism that combines expressiveness and flexibility. This need led to the development of the pre-training and fine-tuning paradigm.

3.3 Pre-training and Fine-tuning

To address the challenges of limited labeled data and generalization issues in GNNs, the pre-training and fine-tuning paradigm, thriving in the natural language processing (NLP) community, has gained widespread adoption in graph representation learning. These approaches involve pre-training models on large-scale graph data, with or without labels, followed by fine-tuning model parameters for diverse downstream tasks. This two-step process improves model initialization, yielding broader optima and enhanced generalization compared to training from scratch. Commonly employed pre-training schemes include Graph AutoEncoders (GAEs) [93], Masked Components Modeling (MCM) [33, 72], Graph Contrastive Learning (GCL) [92, 80], etc. In Section 4, we will delve into a detailed discussion of the pre-training and fine-tuning method, offering a comprehensive picture.

3.4 A Brief History of Prompt Learning

Due to the growing number of model parameters, the conventional "pre-training and fine-tuning" process is evolving into a new approach termed "pre-training, prompting, and predicting" [52]. In this paradigm, instead of manually adapting the pre-trained model for specific downstream tasks, these tasks are reformulated to resemble those addressed during the pre-training phase, aided by prompts. Prompts in NLP take various shapes, including cloze prompts , which complete textual strings like those used in masked language models, and prefix prompts [43, 46], where the input text precedes the answer slot, as employed by autoregressive language models. Some studies involve manually designed templates based on human insights [2, 74, 75], while others explore automated template learning. This includes searching

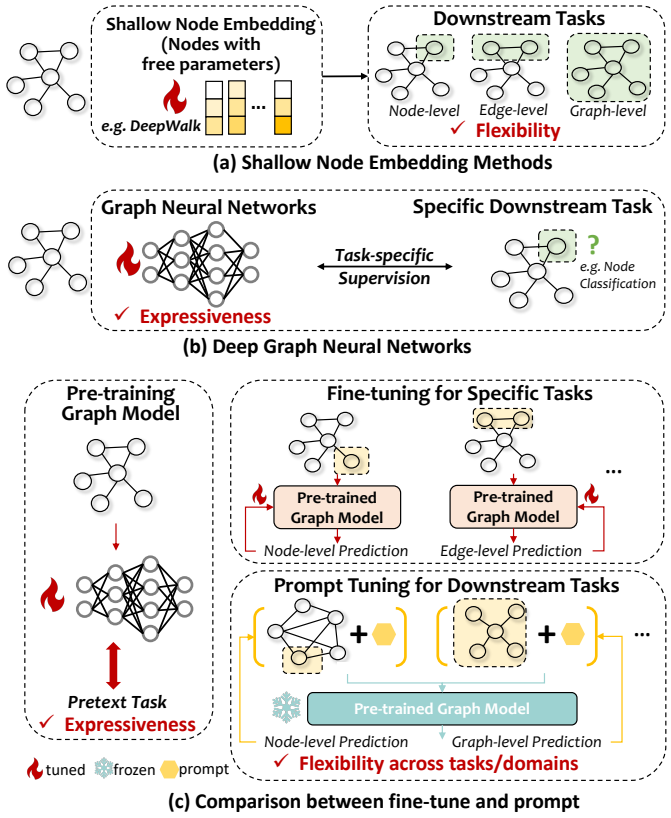


Fig. 5: Our perspective of prompt upon flexibility and expressiveness. (a) Shallow node embedding methods offer flexibility across different downstream tasks but sacrifice expressiveness. (b) GNNs provide expressiveness but require task-specific supervision and constant node features, limiting their flexibility. (c) Prompts enable a balanced approach, achieving both flexibility and expressiveness.

for templates in a discrete space [38, 28, 77, 18] or conducting prompting directly in the embedding space [46, 43, 90, 69]. Such a paradigm enables a single pre-trained model to address a multitude of downstream tasks in an unsupervised manner, which has been widely demonstrated by large language models. In light of this, the application of prompting techniques in the context of graph-based tasks is currently an area of active exploration.

3.5 Why Graph Prompt?

A. Differences between Graph Prompting and Graph Fine-tuning. Compared with traditional "graph pre-training and fine-tuning", which aims to adjust pre-trained graph models for downstream tasks, "graph pre-training and prompting" aims to reformulate downstream tasks as the pertaining task by manipulating input data with graph prompts. In this way, the pre-trained model does not need to be adjusted. A notable difference between graph fine-tuning and prompting is that the former is the training trick on model-level but the latter simulates various transformation strategies on data level.

B. Capability of Making Data Manipulation Learnable. As demonstrated by many works [12, 84], graph prompts are effective in simulating various graph transformations

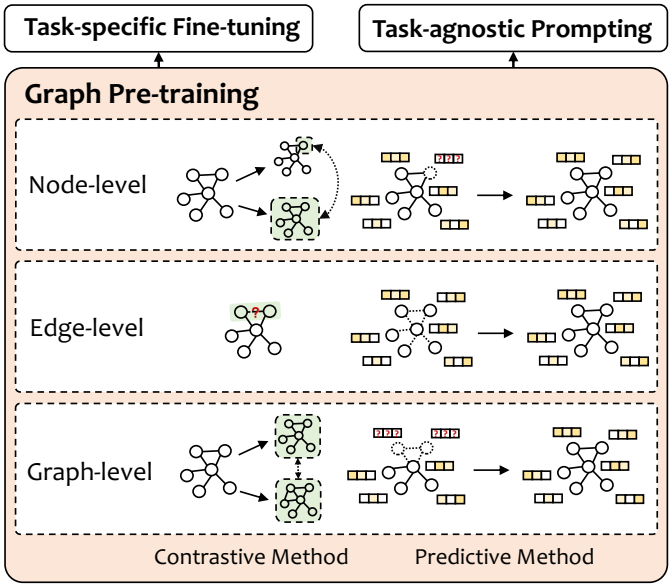


Fig. 6: Graph pre-training methods.

(e.g. node, edge, graph removing or adding, and some node feature operations). For example, let φ^* be a fixed pre-trained graph model, $\mathcal{G}$ and $\mathcal{G}_p^*$ be the original input graph and prompt graph respectively, $\psi(\mathcal{G}, \mathcal{G}_p^*)$ be the hybrid graph integrated by the original graph and the prompt graph. Let $\mathbf{g}(\mathbf{A}$ be the adjacency matrix, and $\mathbf{X})$ be the future matrix of graph nodes. We use $\mathbf{g}(\mathbf{A}, \mathbf{X})$ to denote various graph transformations, then we have the following equation:

$$\varphi^*\left(\psi(\mathcal{G}, \mathcal{G}_p^*)\right) = \varphi^*(\mathbf{g}(\mathbf{A}, \mathbf{X})) + O_{p\varphi}^* \tag{1}$$

where

C. Broader Impact than Fine-tuning. The capability of data manipulation of graph prompts make it be more potential to more application beyond task transferring like fine-tuning. For example, [] use graph prompts to harmony homophilic and heterophilic graphs, make them as one to learn. [] use graph prompt to complete broken ppi to help reseraches ... Beyond this, the inner machnise of graph prompt is very similar to text, image prompt, which means it is potential to unify multi-modal data in one way. Data quality estimation, OOD problem etc.

In summary, if we

4 Pre-training GNNs for Graph Prompting

Graph pre-training is a pivotal step of the pre-training, prompting, and predicting paradigm in graph representation learning. This approach leverages readily available information to encode the inherent graph structure, providing a robust foundation for generalization across diverse downstream tasks. By integrating these pre-training methods into the comprehensive workflow, we offer an exploration of their interplay with the subsequent prompting and predicting phases, shedding light on the strengths and limitations of this holistic approach. This unique perspective distinguishes our survey, framing graph pre-training as an integral part of the broader graph-prompting learning process. To better illustrate the motivation behind the prompting paradigm, we will

now delve into four distinct pre-training strategies within the traditional pre-training and fine-tuning framework.

4.1 Node-level Strategies

Node-level pre-training strategies empower the acquisition of valuable local structure representations that can be transferred to downstream tasks. As shown in Figure 6, these strategies encompass both contrastive and predictive learning methods. In contrastive learning, self-supervised signals typically result from perturbations in the original graph structure or attributes, with the goal of maximizing Mutual Information (MI) between the original and self-supervised views. Noteworthy node-level contrastive methods include those presented in [7, 128, 40, 66, 36, 99]. On the other hand, predictive models focus on reconstructing perturbed data using information from the unperturbed data, as demonstrated in [24, 30, 93, 65, 96, 31]. However, its emphasis on partially semantic topology patterns restricts its ability to capture higher-order information.

4.2 Edge-level Strategies

To enhance performance in tasks such as link prediction, diverse edge-level pre-training strategies have been developed. These strategies excel at capturing node interactions and have undergone extensive exploration. One approach involves discriminating the presence of edges between pairs of nodes, which can be regarded as contrastive methods [41, 58]. Another approach focuses on reconstructing masked edges by recovering the adjacency matrix [86, 44, 62, 26, 42]. Although this pre-training strategy performs admirably in tasks closely related to predicting node relations, it concentrates solely on structural aspects, neglecting the portrayal of node properties, and may encounter challenges when applied to graph-level downstream tasks.

4.3 Graph-level Strategies

The necessity to improve graph-level representations for subgraph-related downstream tasks has prompted the exploration of various graph-level pre-training strategies. Similar to node- and edge-level strategies, these approaches can be broadly categorized into two main groups: graph reconstruction methods, involving the masking of graph components and their subsequent recovery [108, 72], and contrastive methods focused on maximizing mutual information. These contrastive methods target either local patches of node features and global graph features [92, 80, 27, 81, 79], or positive and negative pairs of graphs [115, 85, 88, 70]. While these approaches effectively encode global information and generate valuable graph-level representations, a significant challenge lies in transferring knowledge from a specific pretext task to downstream tasks with substantial gaps, potentially resulting in negative transfer [73]. This can limit the applicability and reliability of pre-trained models and potentially yield less favorable outcomes, even worse than learning from scratch.

4.4 Multi-task Pre-training

Multi-task pre-training accommodates multiple optimization objectives, addressing a broad spectrum of graph-related aspects to enhance generalization while mitigating negative transfer issues. These objectives may encompass various combinations, such as concurrent training of node attribution reconstruction and structural recovery [33, 34, 119, 14]. For example, Hu et al. [33] pre-trained a GNN at both the node and graph levels, enabling the GNN to learn valuable local and global representations simultaneously. Furthermore, some works have employed contrastive learning at different levels [37, 110], or joint optimization of both contrastive loss and graph reconstruction error [45]. However, it is crucial to recognize that multi-task pre-training approaches may face optimization challenges, potentially resulting in suboptimal performance across tasks. As a result, optimizing model performance for each task while mitigating the negative transfer problem remains a significant but unresolved concern.

4.5 Further Discussion

Fortunately, the prompting and predicting paradigm offers a robust solution to the challenges mentioned above. This approach can fully exploit model performance and seamlessly integrate with advanced GNN architectures. Instead of adapting pre-trained GNNs to downstream tasks through objective engineering, this paradigm reformulates downstream tasks into those solved during the pre-training phase using a graph prompt. This innovative strategy effectively bridges the gap between pretext and downstream tasks while sidestepping suboptimal performance pitfalls. Furthermore, in comparison to traditional fine-tuning approaches, the prompting paradigm showcases remarkable flexibility, enabling it to excel in scenarios demanding few-shot or even zero-shot learning, where adapting to new contexts with limited or no labeled data is paramount. In the current landscape marked by surging model volumes and an ever-expanding array of downstream tasks, the ascent of the prompting paradigm represents an irresistible and transformative trend.

5 Prompt Design for Graph Tasks

In this section, we propose a unified view for the graph prompt. As shown in Figure 7, the graph prompt should contain at least three components: prompt tokens with prompt vector; token structures preserving inner correlations of these tokens; and inserting patterns indicating how to integrate the original graph with prompts. Beyond these details, we are particularly interested in the following questions: *Question 1:* How do these works design the graph prompt? *Question 2:* How do these works reformulate downstream tasks to the pre-training tasks? *Question 3:* How do these works learn an effective prompt? and *Question 4:* What are the inner connections of these works, their advantages and limitations? With these questions, we summarize the most representative works published recently and present them in Table 2.

5.1 Prompt Token, Structure, and Inserting Pattern

Question 1: How do They Design A Graph Prompt?

A. Prompt as Tokens. The simplest graph prompt can be treated as some additional features added to the original graph features [130]. For example, given a graph feature matrix $\mathbf{X} = \{x_1, \cdots, x_N\} \in \mathbb{R}^{N \times d}$ where $x_i \in \mathbb{R}^{1 \times d}$ is the feature of i-th node and d is the dimension of the feature

TABLE 2: Summary of existing representative works on graph prompt. $\mathcal{S}$: Subgraph. $V(\mathcal{S})$: Node set within subgraph $\mathcal{S}$. π: Pre-trained parameters. ϕ: Task head parameters. θ: Prompt parameters. $\tilde{\mathbf{s}}$: Filled prompt.

	pre-training task			prompt design			downstream tasks			answering function	
Paper	node	edge	graph	prompt components	inserting pattern	prompt tuning	node	edge	graph	Preset	Learnable
GPPT (KDD 2022 [82])	✗	✓	✗	structure token: $\mathbf{s}_v \in \mathbb{R}^d$ task token: $\mathbf{c}_y \in \mathbb{R}^d$	$\mathbf{s}_{v_i} \leftarrow f_\theta(v_i)$ $\tilde{\mathbf{s}}_{y,v_i} \leftarrow [\mathbf{c}_y, \mathbf{s}_{v_i}]$	Cross Entropy	✓	✗	✗	✓	✗
GPF (arXiv [12])	✓	✓	✓	prompt feature $\mathbf{p} \in \mathbb{R}^d$	$\tilde{\mathbf{s}}_i \leftarrow \mathbf{x}_i + \mathbf{p}$	$\max_{\mathbf{p},\phi} \sum_{(y_i,\tilde{\mathbf{s}}_i)} p_{\pi,\phi}(y_i\|\tilde{\mathbf{s}}_i)$	✗	✗	✓	✗	✓
All in One (KDD 2023 [84])	✗	✗	✓	prompt token: $\mathcal{P} = \{\mathbf{p}_1, ..., \mathbf{p}_{\|\mathcal{P}\|}\}$ token structure: $\{(\mathbf{p}_i, \mathbf{p}_j) \| \mathbf{p}_i, \mathbf{p}_j \in \mathcal{P}\}$	$w_{ik} \leftarrow \sigma(\mathbf{p}_k \cdot \mathbf{x}_i^T)$ if $\sigma(\mathbf{p}_k \cdot \mathbf{x}_i^T) > \delta$ else 0 $\tilde{\mathbf{s}}_i \leftarrow \mathbf{x}_i + \sum_{k=1}^{\|\mathcal{P}\|} w_{ik}\mathbf{p}_k$	Meta-Learning	✓	✓	✓	✓	✓
GraphPrompt (WWW 2023[55])	✗	✓	✗	prompt token: $\mathbf{p}_t \in \mathbb{R}^d, t \in \mathcal{T}$ structure token: $\mathbf{s} \in \mathbb{R}^d$ task token: $\mathbf{c}_y \in \mathbb{R}^d$	$\tilde{\mathbf{s}}_i^t \leftarrow \text{Readout}(\{\mathbf{p}_t \odot f_\pi(v) \| v \in V(\mathcal{S}_i)\})$ $\mathbf{c}_y \leftarrow \text{Mean}(\{\tilde{\mathbf{s}}_j^t \| y_j = y\})$	$\min_{\mathbf{p}_t} -\sum_{(y_i,\mathcal{S}_i)} \ln \frac{\exp(\text{sim}(\tilde{\mathbf{s}}_i^t, \mathbf{c}_{y_i})/\tau)}{\sum_{y \in Y} \exp(\text{sim}(\tilde{\mathbf{s}}_i^t, \mathbf{c}_y)/\tau)}$	✓	✗	✓	✓	✗
PGCL (arXiv [21])	✗	✗	✓	semantic token: $\mathbf{p}^s \in \mathbb{R}^d$ contextual token: $\mathbf{p}^c \in \mathbb{R}^d$	$z_x^{ps} = z_x^s \odot \mathbf{p}^s$ $z_x^{pc} = z_x^c \odot \mathbf{p}^c$	$\min -\sum_{(v,a,b)\in\mathcal{T}} \log \frac{\exp(\text{sim}(z_v^p, z_a^p)/\tau)}{\sum_{u\in\{x,y\}} \exp(\text{sim}(z_v^p, z_u^p)/\tau)}$	✓	✓	✓	✓	✗
PRODIGY (NeurIPS 2023 [35])	✗	✓	✗	data graph: $\mathcal{G}^D \sim \oplus_{i=1}^K \tilde{\mathcal{N}}(\mathcal{V}_i, \mathcal{G})_\rangle$ task graph: $\mathcal{G}^T$	$\tilde{\mathbf{s}}_i \leftarrow f_{\pi^T}^T(\mathcal{G}^T \| f_{\pi^D}^D(\mathcal{G}^D))$	Fixed	✓	✓	✓	✓	✗
SGL-PT (arXiv [129])	✓	✗	✓	prompt token: one vector for each graph	connect to all nodes in the graph	contrastive loss and reconstruction loss	✓	✗	✗	✓	✗
GPF-Plus (NeurIPS 2023 [13])	✓	✓	✓	prompt features $\mathbf{p}_1, \cdots, \mathbf{p}_k \in \mathbb{R}^d$	$\tilde{\mathbf{s}}_i \leftarrow \mathbf{x}_i + \sigma(\mathbf{p}_1, \cdots, \mathbf{p}_k)$	$\max_{\mathbf{p},\phi} \sum_{(y_i,\tilde{\mathbf{s}}_i)} p_{\pi,\phi}(y_i\|\tilde{\mathbf{s}}_i)$	✗	✗	✓	✗	✓
DeepGPT (arXiv [78])	✓	✓	✓	prompt token: $\mathbf{p} \in \mathbb{R}^d$ prefix token: $\mathbf{P} \in \mathbb{R}^{\|\mathcal{P}\|\times d}$	$\tilde{\mathbf{x}}_i \leftarrow \mathbf{x}_i + \mathbf{p}$ $\tilde{\mathbf{s}}_i \leftarrow f_{\mathbf{P},\pi}(G, \tilde{\mathbf{x}}_i)$	$\min_{\mathbf{p},\mathbf{P},\phi} \sum_{(y_i,v_i)} \mathcal{L}(p_\phi(\tilde{\mathbf{s}}_i), y_i)$	✗	✗	✓	✗	✓
ULTRA-DP (arXiv [4])	✗	✓	✗	prompt token: $\mathbf{p}_i = \mathbf{p}^{\text{task}} + w^{\text{pos}}\mathbf{p}_i^{\text{pos}}$, $\mathbf{p}_i^{\text{pos}}$ denotes v_i's positional embedding	create a virtual node v_i^p for target node v_i, $G' \leftarrow (\mathcal{V} \cup \{v_i^p\}, \mathcal{E} \cup \{(v_i^p, v_i)\}, \mathbf{X} \cup \{\mathbf{p}_i\})$	Multi-task-Learning	✓	✗	✗	✗	✓
HetGPT (arXiv [59])	✗	✓	✗	prompt token: $\mathcal{F} = \{\mathbf{f}_i^A\}_{i=1}^K$ for $A \in \mathcal{A}$ task token: $\mathbf{c}_y \in \mathbb{R}^d$	$\tilde{\mathbf{s}}_i^A \leftarrow \mathbf{x}_i^A + \sum_{k=1}^K w_{ik}\mathbf{f}_k^A$, $\mathbf{z}_i = f_\pi(G, \tilde{\mathbf{s}}_i^A)$	$\min_{\mathbf{C},\mathcal{F}} -\sum_{(y_i,v_i)} \log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{c}_{y_i})/\tau)}{\sum_{y\in\mathbf{Y}} \exp(\text{sim}(\mathbf{z}_i, \mathbf{c}_y)/\tau)}$	✓	✗	✗	✓	✗
SAP (arXiv [20])	✓	✗	✗	task token: $\mathcal{P} = \{\mathbf{c}_y\}_{y\in\mathbf{Y}}$ structure token: $\mathcal{W} = \{(v_i, c_j)\}_{v_i\in\mathcal{V}, c_j\in\mathcal{P}}$	$G' \leftarrow (\mathcal{V} \cup \mathcal{P}, \mathcal{E} \cup \mathcal{W})$ $\mathbf{Z}^{(1)} = \text{MLP}_{\pi'}(\mathbf{X})$ $\mathbf{Z}^{(2)} = \text{GNN}_{\pi''}([\mathbf{X}, \mathbf{P}], [\mathbf{A}, \mathbf{W}])$	$\min_{\mathbf{W}} -\sum_{(y_i,v_i)} \log \frac{\exp(\text{sim}(\mathbf{z}_i^{(1)}, \mathbf{z}_{y_i}^{(2)})/\tau)}{\sum_{y\in\mathbf{Y}} \exp(\text{sim}(\mathbf{z}_i^{(1)}, \mathbf{z}_y^{(2)})/\tau)}$	✓	✗	✓	✓	✗
VNT (KDD 2023 [87])	✓	✓	✗	$\boldsymbol{P} = [\boldsymbol{p}_1; \ldots; \boldsymbol{p}_P]$, $\boldsymbol{p}_p \in \mathbb{R}^F$	$[E^1 \| Z^1] = L^1([E^0 \| P]) \in \mathbb{R}^{(V+P)\times F}$	Cross Entropy	✓	✗	✗	✗	✓

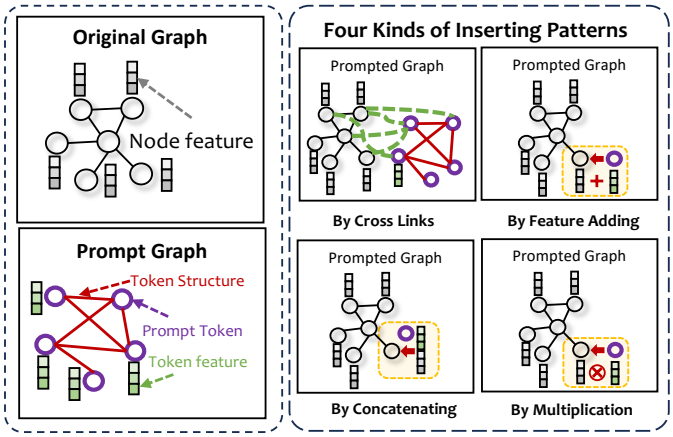


Fig. 7: Prompt Tokens, Structures, and Inserting Patterns.

space. Fang et al. [12] and Shirkavand and Huang [78] treat the basic prompt as a learnable vector $p \in \mathbb{R}^{1\times d}$, which can be added to all node features and make the manipulated feature matrix be $\mathbf{X}^* = \{x_1 + p, \cdots, x_N + p\}$. In this way, we can use the reformulated features to replace the original features and process the graph with pre-trained graph models. A later work [13] further extends one prompt token to multiple tokens and makes the performance better. PGCL [21] design a prompt vector for semantic view and another prompt vector for contextual view, then they add these prompt vectors to the graph-level representations by element-wise multiplication. A similar prompt design is also adopted in VNT [87]. The difference is that their inserting pattern does not add the prompt token to the original graph feature but concatenates the prompt token with the original node set and tries to integrate them by the self-attention function in a graph transformer. In GraphPrompt [55], the prompt token is similar to the format defined by Fang et al. [12]. The difference thing is that previous work designed the prompt tokens in the initial feature space, while this method assumes the prompt in the hidden layer of the graph model. Usually, the hidden size will be smaller than the original feature, making their prompt token shorter than the previous one. Another different thing is that the graph prompt here is used to assist graph pooling operation (a.k.a Readout). For example, given the node set $\mathcal{V} = \{v_1, \cdots, v_{|\mathcal{V}|}\}$, the embedding of node v is $\mathbf{h}_v \in \mathbb{R}^{1\times d}$, a prompt token $\mathbf{p}_t \in \mathbb{R}^{1\times d}$ specified to task t is inserted to the graph nodes by the element-wise multiplication ($\otimes$): $\mathbf{s}_t = \text{Readout}(\{\mathbf{p}_t \otimes \mathbf{h}_v : v \in \mathcal{V}\})$. Similarly, SGL-PT [129] creates a prompt token to connect to all nodes in the graph.

The prompt token preserves a global perception of the graph and can assist their global branch of the pre-training tasks, which can be also treated as a pooling strategy. Aiming at aligning node classification and link prediction, GPPT [82] defines graph prompts as additional tokens that contain task tokens and structure tokens. Here the task token refers to the description of the downstream node label to be classified and the structure token is the representation of the subgraph surrounding the target node. By this means, predicting node v's label can be reformulated to predict a potential link between node v's structure token and the label task token. Aiming at the node classification task, ULTRA-DP [4] creates a prompt token for each target node, where the token feature is the weighted sum of position embedding of the target node and a task embedding of the pre-training task. HetGPT [59] design a prompt with node tokens and class tokens, which are organized in a similar way to GPPT. The difference is that they also add a type-specific feature token to make graph prompts sensitive to different node types, by which they can extend existing graph prompts to heterogeneous graphs.

B. Prompt as Graphs. The graph prompt in All in One [84] is an additional subgraph that can be learned by efficient tuning. The prompt tokens are some additional nodes that have the same size of node representation as the original nodes. They assume the prompt tokens should be in the same semantic space as the original node features so that we can easily manipulate node features with these tokens. The token structures include two parts. The first part is the inner links among different tokens, and the second part is the cross-links between the prompt graph and the original graph. These links can be pre-calculated by the dot product between one token to another token (inner links) or one token to another original node (cross links). The inserting pattern is to add the prompt graph to the original graph by the cross-links and then treat the combined graph as a new graph, and send it to the pre-trained graph model to get the graph-level representation. The prompt graph in PRODIGY [35] includes data graphs and a task graph. The data graphs can be treated as subgraphs surrounding the target nodes (for node classification task), node pairs (for edge classification task), or just denoted as the graph classification instance. Here the prompt tokens and prompt structures are just the same as in the original graph. The task graph contains data tokens and label tokens where each data token connects to one data graph and is further connected by label tokens. Unlike previous works that aim at reformulating downstream tasks to the pre-training task by prompting the downstream data, PRODIGY leverages the prompt graph to unify all the upstream and downstream tasks. Their pre-training strategy is a set of tasks including neighboring matching and label matching, which can be reformulated as predicting the similarity between the data token and the label token in the prompt graph. SAP [20] also connects several prompt tokens (each token corresponds to one class) to the original graph by cross-links defined in All in One. The difference is that their prompting task is a node-level contrastive task, in which they use MLP to encode the node features as the first view and they use a GNN to encode the prompted graph as the second view, which is consistent with their pre-training task.

5.2 Aligning Tasks by Answering Function

Question 2: How do they reformulate downstream tasks to the pretext?

A. Handling Different Level Tasks. All in One [84] pre-trains the graph model via graph-level contrastive learning. The pre-training task aims to learn a robust graph encoder over different graph views generated by the perturbations of the original graph. To reformulate various graph tasks to this graph-level pretext, they first unify node-level, edge-level, and graph-level tasks to the graph-level task by induced subgraphs, which are also introduced in PRODIGY [35, 55, 21]. Then they claim that the graph prompt added to the original graph is in nature the simulation of any graph operations such as node feature masking, node or edge perturbations, subgraph removing, etc. With this opinion, we just need to add an appropriate graph prompt to downstream graph datasets then it will be promising to further reformulate the downstream task to the pretext.

B. Learnable Answering Function. To output the results of downstream tasks, Sun et al. [84] design two types of answering functions. The first one is a learnable task head (such as an MLP mapping function) that can be easily tuned with very limited data. It takes the graph-level representation generated by the pre-trained graph model and then outputs the downstream result. For example, if the downstream is a three-class node classification, we can simply use a dense layer with three hidden units to connect the graph representation, which is generated by the pre-trained model on the combined graph with node included graph and a graph prompt. In this case, both the graph prompt and the task head are tunable, so we can adjust them alternately. Similar learnable answering functions are also adopted in other works like [12, 13, 87]. The good point is that they are very easy to align two tasks, however, it also increases the tuning workflow.

C. Hand-crafted Answering Function. To further reduce the tuning burden, All in One [84] also proposes a second answering function, which is hand-crafted without any trainable task head. For example, for a node classification task, we can set up K unique sub-prompts, each aligning with a different node type, where K represents the total number of node categories. If the pre-training involves a task like GraphCL [115], which aims to maximize similarity at the graph level between pairs of graph views, then the target node can be classified with label $\ell, (\ell = 1, 2, \cdots, K)$ if the ℓ-th graph most closely resembles the original node-inclusive graph. Similarly, GPPT [82] and HetGPT [59] use link prediction as their pre-training task and reformulate downstream node classification by unifying it as the same task template. For example, by treating the node label as an additional token, we can use the pre-trained model to directly output the possibility of an edge between the label token and the target node. The pre-training strategy in SAP [20] is to compare node representations from two graph views, the first of which is generated by node feature encoding and the second of which is encoded by a graph model. To this end, their prompt tokens denote class information and they compare node representation with each class token to find the class with the largest similarity as the inference results. By designing a unified task template, PRODIGY [35] uses a

hand-crafted graph prompt to describe all node, edge, and graph classification tasks by predicting the link between data tokens and label tokens. Liu et al. [55] extend the link prediction task as graph pair similarity and treat it as their pre-training task, to align the downstream node classification and graph classification task, they design a unified answering template making the downstream side aligned with the pre-training side. Specifically, given a triplet of induced graphs (g_1, g_2, g_3) where g_1 and g_2 have the same label, g_1 and g_3 have different labels. In particular, when the target task is node classification, the induced graph refers to the contextual subgraph of the target node. The unified answering template is defined as $\text{sim}(g_1, g_2) > \text{sim}(g_1, g_3)$.

5.3 Prompt Tuning

Question 3: How do They Learn A Graph Prompt?

A. Meta-Learning Technique. To learn appropriate prompts, Sun et al. [84] leverage meta-learning techniques (such as MAML[16] model) to obtain a robust starting point for the prompt parameters. Since the support set and query set include various graph tasks (such as node classification, link prediction, graph classification, etc), the learned graph prompt is expected to be more general on various downstream tasks.

B. Task-specific Tuning. Besides All in One [84], which aims to learn a general prompt on various downstream tasks, there are also some works that target specific downstream tasks such as graph classification. In this case, the prompt tuning can be more task-directed. For example, GPF [12] aims at a graph classification task, so it just needs to tune the prompt token $\mathbf{p}$ and the task head ϕ by maximizing the likelihood of predicted correct graph labels given the prompted graph representation $\tilde{\mathbf{s}}_i$ from the pre-trained graph model π. In this case, the task head tuning and the prompt tuning share the same objectives, which can be formulated by: $\max_{\mathbf{p},\phi} \sum_{(y_i,\tilde{\mathbf{s}}_i)} p_{\pi,\phi}(y_i|\tilde{\mathbf{s}}_i)$.

C. Tuning in Line with Pretext. Intuitively, prompting aims at reformulating downstream tasks to the pre-training task. Therefore, it would be more natural if the prompt tuning shares the same objective with the pre-training task. As suggested in GraphPrompt [55], the authors use a similar loss function to learn prompts. Similarly, GPPT [82] and VNT [87] adopt the same loss function (Cross-Entropy) as their link prediction and node classification tasks, respectively. HetGPT [59] and SAP [20] use a node-level contrastive loss to learn their prompt tokens because their pre-training task is also conducted by the same contrastive task (node pair comparison). PGCL [21] introduces graph-level loss to align with the pre-training task. ULTRA-DP [4] develop two pre-training tasks including edge prediction and neighboring prediction, each of which corresponds to one task embedding. In the pre-training phase, they randomly select a task and then integrate specific task-related embeddings into the prompt tokens. These learnable task embeddings are then trained with the graph model.

5.4 Further Discussion

Question 4: What are Their Connections, Pros and Cons?

The good point of GPF [12] is that they propose a very simple prompt that can be easily used in various pre-training tasks and downstream tasks. However, a single prompt token added to all nodes is very limited in expressiveness and generalization. A potential solution is to learn an independent prompt token for each node, which means one node corresponds to one prompt token, but this will cause low efficiency in parameters. To this end, we can train K-independent basis vectors and use them to compound each node token (GPF-Plus [13]). This improvement makes their work have more similar insights with All in One [84].

HetGPT [59] extends prompt tokens to type-specific format, which can deal with graph prompting in heterogeneous data. However, they can only deal with node classification tasks. To this end, GraphPrompt [55] reformulates link prediction to graph pair similarity task. It is worth noticing that the role of their prompt token is very similar to the project vector in the graph attention network. There are also some attention-based graph-pooling methods, which share the same motivation. The difference is that the authors claim the graph-pooling component in the pre-training stage might not fit other downstream tasks, thus needing additional prompts to redirect the graph-pooling component in the graph model.

GPPT [82] represents a specific instance within the broader framework of All in One [84]. For instance, if we minimize the prompt graph to isolated tokens that correlate with node classes and substitute the resulting graphs with a complete graph, the All in One prompt structure can be simplified into the GPPT format. This allows for the utilization of edge-level pretexts in node classification tasks within the GPPT framework. The shortcoming of GPPT might be that it is restricted to binary edge prediction pretexts and is solely effective for node classification in downstream tasks. In comparison, frameworks like GraphPrompt and All in One are designed to accommodate a wider array of graph-related tasks, extending beyond just node-level classification. The good point is that when adapting models for different tasks, GraphPrompt, GPF, and GPF-Plus often require the tuning of an extra task-specific module. In contrast, All in One, and GPPT utilize task templates that focus more on the manipulation of input data and are less dependent on the specifics of downstream tasks.

Intuitively, the data graphs, one of the components in PRODIGY [35], are very similar to the induced graph in All in One and GraphPrompt. The pre-training task in PRODIGY can be seen as predicting a link between the data token and the label token, which shares a similar insight with GPPT. The good thing is that their prompts have no trainable parameters, which means they do not need to tune the prompt and are more efficient in the in-context learning area. PRODIGY does not need any tuning work and can be used in knowledge transferring between different datasets. However, a non-tunable prompt is usually not flexible enough, which might also limit the generalization of the pre-trained model when the downstream tasks to be transferred are located in a different domain from the pre-training one. In contrast, ULTRA-DP [4] tune prompt both in the pre-training stage and the downstream tasks. It first put the prompt tuning work in the pre-training stage to obtain the task embeddings, which are one of the main components in their prompt. Then they use these task embeddings to initialize a downstream prompt. Intuitively, their prompts

are not used to reformulate downstream tasks to the pretext. Instead, these prompt tokens are used to select suitable pre-training tasks from a task pool to fit the downstream task. It is still an interesting question of how to achieve the optimal balance given efficiency, generalization, and the flexibility of prompt.

Compared with other works that usually define clear inserting patterns, VNT [87] concatenates prompt tokens with the original node set and then puts all of them into the graph transformer. Actually, the graph transformer will leverage a self-attention function to further calculate the correlations among them, which can also be treated as a variant of inserting patterns defined in All in One. The good thing is that we do not need to design a threshold to tailor the connection but the shortcoming is that it can only use a graph transformer as its backbone and can not applied to more message-passing-based graph models. In addition, there are also some more advanced variants of graph transformers requiring additional position embedding as one of their input. However, the prompt tokens in VNT have no clear inserting links to the original graph, which might not make it easy to apply existing position encoding approaches for these graph transformer variants.

6 Graph Prompting in Multi-Modal and Multi-Domain Areas

6.1 Multi-Modal Prompting with Graphs

The fusion of images, sound, and text has been widely studied and achieved remarkable success. However, most of these modalities are described by linear data structure. In our real-world life, there are more kinds of data in non-linear structures like graphs. How to connect these linear modalities (e.g. text, images, sound, etc) to the non-linear modalities (e.g. graphs) has become a very attractive research topic because it is a bigger move towards artificial general intelligence. Unfortunately, reaching this vision is very tough. Currently, we only see some hard progress in the fusion of text and graphs, especially in the text-attributed graphs. With the help of recent large language models, the fusion of text and graph has achieved even more notable performance. Since there have already been some informative surveys on this topic, we next briefly discuss some representative works that are closely related to *prompt techniques*. We suggest readers refer to the mentioned literature [49, 51] to require more detailed information further.

A. Prompt in Text-Attributed Graphs. Wen and Fang [101] ventured into enhancing text classification in scenarios with limited data resources by the proposed model, Graph-Grounded Pre-training and Prompting (G2P2). Their work identifies the issue of insufficient labeled data in supervised learning and proposes a solution leveraging the inherent network structure of text data, such as hyperlinks or citation networks. G2P2 utilizes text-graph interaction strategies with contrastive learning during the pre-training phase, followed by an inventive prompting technique during the classification phase, demonstrating notable effectiveness in zero- and few-shot text classification tasks. Zhao et al. [126] focused on molecule property prediction, a field grappling with the scarcity of labeled data. Their study introduces an integrated graph-text model to enhance prompt-based molecule task learning in a zero-shot context. This model employs generalized position embedding and decouples encoding of the graph from task prompt, enhancing its generalization capability across novel tasks. Li and Hooi [48] explored node classification within the framework of multi-modal data (text and graph), particularly focusing on limited-label scenarios. Unlike traditional works that usually feed pre-computed text features into graph neural networks, they incorporate raw texts and graph topology by a hand-crafted language prompt template into the model design.

B. Large Language Models in Graph Data Processing. Chen et al. [6] explored the potential of Large Language Models (LLMs) in graph node classification tasks. They investigated two pipelines: LLMs-as-Enhancers, which enhances node text attributes using LLMs followed by predictions via Graph Neural Networks (GNNs), and LLMs-as-Predictors, which directly employs LLMs as standalone predictors. Their empirical evaluations revealed that deep sentence embedding models and text-level augmentation through LLMs effectively enhance node attributes, while LLMs also show promise as standalone predictors, albeit with concerns about accuracy and test data leakage. Fatemi et al. [15] conducted a comprehensive study on encoding graph-structured data for consumption by LLMs. Their findings include the influence of the graph encoding method, the nature of the graph task, and the structure of the graph on LLM performance. They demonstrated that simple prompts are most effective for basic graph tasks and that graph encoding functions significantly impact LLM reasoning. Their experimental setup introduced modifications to the graph encoding function, revealing improvements in performance and demonstrating the effect of model capacity on graph reasoning ability. Jin et al. [39] introduced an innovative approach to pre-train language models on networks rich in text. Their framework, named PATTON, focuses on integrating the intricacies of textual attributes with the underlying network structure. They developed two novel pretraining strategies: one concentrating on the context within networks for masked language modeling, and the other on predicting masked nodes, thus capturing the interplay between text and network structure. The effectiveness of PATTON was validated through various experiments, showcasing its superior performance over traditional text/graph pretraining methods in diverse tasks such as document classification, retrieval, and link prediction in different domain datasets. This approach signifies a shift in pretraining methodologies, emphasizing the synergy between textual data and network context. Wang et al. [100] proposed a Knowledge Graph Prompting (KGP) method to enhance LLMs for multi-document question answering (MD-QA). They created a knowledge graph over multiple documents, with nodes representing passages or document structures and edges denoting semantic/lexical similarity. The LM-guided graph traverser in KGP navigates the graph to gather supporting passages, aiding LLMs in MD-QA. Their experiments indicated that the construction of KGs and the design of the LM-guided graph traverser significantly impact MD-QA performance.

C. Multi-modal Fusion with Graph and Prompting. The integration of multi-modal data using graph and prompting techniques has seen remarkable progress in recent years. For example, Edwards et al. [11] propose SynerGPT in the

field of drug synergy prediction. This model leverages a transformer-based approach, uniquely combining in-context learning with genetic algorithms to predict drug synergies. In the area of vision-language models, Li et al. [47] develop GraphAdapter, a prompt-based strategy that utilizes an adapter-style tuning mechanism, bringing together textual and visual modalities through a dual knowledge graph. Liu et al. [53] extend work multi-modal fusion into molecular science with their proposed GIT-Mol. A large language model integrates graph, image, and textual data with the help of prompt, offering substantial improvements in various tasks like molecule generation and property prediction. Although much effort has been made in the past few years, the academic is still trying hard to find better solutions to integrate text and graphs via text-attributed graphs or knowledge graphs. There is still a very large imagination in the fusion of more kinds of modalities.

6.2 Graph Domain Adaptation with Prompting

The field of graph domain adaptation has seen significant advancements, particularly with the integration of prompting techniques. However, graph domain adaptation is still not a well-solved problem because there exist at least two challenges: The first one is how to align semantic spaces from different domains. The second one is how to identify structural differences.

A. Semantic Alignment. In particular, All in One [84] extends the "pre-training and fine-tuning" workflow with multi-task prompting for GNNs, unifying prompt formats, and introducing meta-learning for prompt optimization. To make the graph model adaptive to different graph domains, they first reveal that the graph prompt in nature can be seen as graph operation and then they use graph prompt to manipulate different domain graph datasets. GraphControl [130] introduces a unique deployment module inspired by ControlNet, effectively integrating downstream-specific information as conditional inputs to enhance the adaptability of pre-trained models to target data. This approach aligns input space across various graphs and incorporates unique characteristics of the target data. Zhang et al. [123] presents a pre-training model for knowledge graph transfer learning. This model uses a general prompt-tuning mechanism, treating task data as a triple prompt, enabling flexible interactions between task KGs and task data. Yi et al. [114] combines personalized graph prompts with contrastive learning for efficient and effective cross-domain recommendation, particularly in cold-start scenarios. A representative work is proposed by Liu et al. [50], in which they describe graph nodes from different domains by language and then use LLM to get a textual embedding. However, this work needs the semantic name of each feature while sometimes graph features are usually latent vectors without clear semantic meaning.

B. Structural Alignment. Cao et al. [3] analyze the feasibility between different graph datasets and found that the structural gap holds the upper bound of various graph model transferability. GraphGLOW [127] addresses the limitations of existing models that operate under a closed-world assumption, where the testing graph is identical to the training graph. This approach often leads to prohibitive computational costs and overfitting risks due to the need for training a structure learning model from scratch for each graph dataset. To this end, it coordinates a single graph-shared structure learner with multiple graph-specific GNNs to capture generalizable patterns of optimal message-passing topology across datasets. The structure learner, once trained, can produce adaptive structures for unseen target graphs without fine-tuning, thereby significantly reducing training time and computational resources. AAGOD by Guo et al. [23] proposes a data-centric framework for OOD detection in GNNs. They use a parameterized amplifier matrix, which is treated as a prompt, to superimpose on the adjacency matrix of input graphs. Inspired by prompt tuning, Shirkavand and Huang [78] propose DeepGPT for graph transformer models. They add prompt tokens to the input graph and each transformer layer. By updating the prompt tokens, they can efficiently tune a graph model on different graph datasets. A recent work [125] leverages the graph prompt capability in manipulating graph data and tries to integrate multiple graph datasets in the graph pre-training stage, the results from which strongly suggest the huge potential of transferring preserved knowledge from one domain to another.

7 Potential Applications

With the widespread utilization of networks as a data modeling structure for representing diverse relational information across social, natural, and academic domains, the graph prompt mechanism exhibits substantial potential for a wide range of real-world applications. In this section, we explore the potential applications of graph prompting in online social networks, recommender systems, knowledge management, and biology.

Online Social Networks. Online social platforms consist of users who can be represented as nodes, and their social connections form online social networks (OSNs). Previous research has investigated the potential of prompting mechanisms in identifying fake news within OSNs to prevent malicious attacks [105]. Specifically, they employ textual prompts applied to pre-trained language models (PLMs) to distill general semantic information. By combining this semantic signal with the dynamics of information propagation within social networks, improved classification performance can be achieved. While the use of tailored textual prompts for PLMs has been studied, the application of graph prompting mechanisms within social networks is still under-explored. In the future, it is promising to directly apply prompt tuning techniques to social networks, utilizing few-shot labels for tasks such as fake news detection or anomaly detection [103, 23], where the labeling process is laborious and requires domain expertise. By incorporating prompts directly within social networks, this approach can address the scarcity of labeled data and enhance the security and trustworthiness of online social networks. Some recent work [5] indicates that the graph prompt presents a significant performance in the analysis of temporal interactions, which is promising when applied to online social networks.

Recommender Systems. E-commerce platforms provide a valuable opportunity to leverage recommender systems

for enhancing online services. While prompt tuning in recommender systems has received limited research attention, it holds significant potential [114, 113, 107, 25]. In [114], the graph prompt tuning technique is applied to cross-domain recommendation scenarios to address the challenges of domain adaptation. Specifically, when applying a pre-trained recommendation model to the target domain, extra prompt nodes are introduced to achieve both efficient and effective domain recommendation. Meanwhile, Yang et al. [113] propose personalized user prompts to bridge the gap between contrastive pretext [104, 116] to downstream recommendation task. They design different kinds of personalized prompts, in combination with pre-trained user embeddings to facilitate dynamic user representations, leading to more accurate and personalized recommending results. In the future, further exploration into the integration of graph prompt tuning within recommender systems can be conducted to enhance recommendation performance, personalization, and adaptability across different domains.

Knowledge Management. There are two branches of research focused on performing prompting on knowledge graph (KG) for improved knowledge management. The first branch involves direct prompting on knowledge graphs using a pre-trained KG model to facilitate knowledge transfer, enabling better generalization across different KG data and tasks [123]. The second branch explores the combination of grounded knowledge from KGs with the cognitive abilities of LLMs [100, 89, 71, 64, 122, 63] to improve performance in downstream tasks. In the first research line, Zhang et al. [123] proposed a structure pre-training and prompt tuning approach to realize knowledge transfer. They designed specific pre-training objectives to obtain a powerful KG model. Subsequently, a general prompt tuning technique was employed to facilitate knowledge transfer between task-specific KGs and data. In the second research line, prompt tuning techniques are adopted to combine the grounded knowledge inherent in KGs with LLMs for enhancing downstream tasks. For example, in [89], a novel graph neural prompting method was introduced to adapt KGs for LLMs by distilling valuable knowledge from KGs in a time- and parameter-efficient manner. Future research can further explore this trend by implementing more efficient graph prompting methods to fully distill beneficial knowledge from KGs to assist LLMs on diverse downstream tasks.

Biology. Molecules can be represented naturally as graphs, where atoms serve as nodes and chemical bonds act as edges [72]. Such graph modeling provides a basis for applying graph representation learning methods to perform tasks such as molecular property prediction, thereby benefiting scientific research and discovery [72, 56, 126, 11]. Previous research on graph representation learning in the molecular domain followed a task-specific approach. It involved training individual models tailored to specific molecule datasets [72, 80], lacking the generalization ability within the domain. Though there exist works that explored the utilization of LLMs (or LMs) as a universal tool for understanding molecules [68, 50], these approaches primarily rely on regular texts to describe molecules, overlooking the inner graph structures within molecules [124]. Meanwhile, some recent works have focused on investigating the co-modeling of graphs and languages to preserve both structural dependencies and achieve generalization abilities across tasks and datasets, even under few-shot or zero-shot settings [53, 126, 11]. For instance, in [53], the authors employed LoRA [32] to efficiently adapt to downstream tasks. Following recent work on graph prompt[19], which has achieved remarkable results in protein multimer structure prediction, we believe that graph prompting techniques can also be adopted to achieve more efficient task adaptation within the molecular domain.

8 ProG: A Unified Library for Graph Prompting

An indispensable component for fortifying the graph prompting ecosystem is a well-crafted tool. Despite the plethora of tools proposed for generalized graph learning, a notable absence persists in the realm of libraries dedicated to graph prompt functionalities. Addressing this gap, we are introducing `ProG` (Prompt Graph), an open-source, unified library meticulously designed to cater to the specific needs of graph prompting. This initiative promises to significantly enhance the landscape of graph-based applications by providing a versatile and comprehensive resource for researchers and practitioners alike.

`ProG` is a PyTorch-based library designed to facilitate single or multi-task prompting for pre-trained GNNs. The architecture is illustrated in Figure 8. It seamlessly integrates several widely used datasets in the graph prompt evaluation, including Cora, CiteSeer, Reddit, Amazon, and Pubmed etc. The tool is equipped with essential evaluation metrics such as Accuracy, F1 Score, and AUC score, commonly employed in various graph prompt-related tasks. Notably, `ProG` incorporates state-of-the-art methods like All in One [84], GraphPrompt [55], GPPT [82], GPF [12], and GPF-Plus [13], and it continues integrating more graph prompt models. In summary, `ProG` offers the following key features:

- **Quick Initiation.** All models are implemented within a consistent environment, accompanied by detailed demos, ensuring a swift initiation for newcomers.
- **Fully Modular.** `ProG` adopts a modular structure, empowering users to customize and construct models as needed.
- **Easy Extendable.** `ProG` is designed for seamless extension to encompass additional methods and a broader spectrum of downstream tasks, adapting to evolving research needs.
- **Standardized Evaluation.** `ProG` establishes a uniform set of evaluation processes, promoting equitable performance comparisons across models.

For additional information and access to the library, please visit the website of our library[5]. Additionally, we have curated a GitHub repository[6], serving as a centralized resource for the latest advancements in graph prompt learning. This repository includes a list of research papers, benchmark datasets, and available codes, fostering an environment conducive to ongoing research in this dynamic field. Regular real-time updates ensure that the repository remains current with emerging papers and associated codes.

5. https://github.com/sheldonresearch/ProG
6. https://github.com/WxxShirley/Awesome-Graph-Prompt

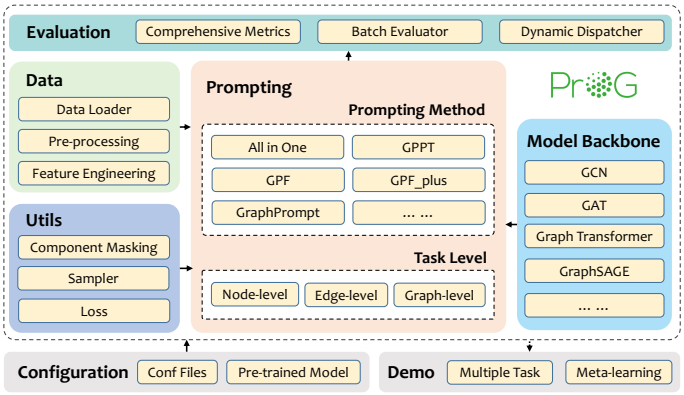


Fig. 8: The architecture of `ProG`.

9 Challenges and Future Directions

9.1 Current Challenges

Graph prompt learning has made significant research progress, but it still encounters several challenges. In this subsection, we will discuss current challenges in detail.

Inherent Limitation within Graph Models Prompts, originated from the NLP field [2, 43, 52], serve as a means to unlock the potential of pre-trained language models for adapting to downstream tasks. This in-context learning ability emerges when the scale of model parameters reaches a certain level. For instance, widely known LLMs typically possess billions of parameters. With such a powerful pre-trained model, a simple textual prompt can distill specific knowledge for downstream tasks. However, when it comes to prompting on graph tasks, the conditions become more intractable since pre-trained graph models have significantly fewer parameters, making it challenging to harness their full downstream adaptation potential.

Intuitive Evaluation of Graph Prompt Learning In the NLP field, prompts are typically in a discrete textual format [2, 71], allowing for intuitive understanding, comparison, and explanation. However, existing graph prompts are represented as learnable tokens [82, 13, 12, 87, 59] or augmented graphs [84, 35, 20]. Such format poses challenges in intuitively understanding and interpreting graph prompts, as they lack a readable design. As a result, the effectiveness of prompts can only be evaluated based on downstream tasks, limiting efficient and comprehensive performance comparison among different kinds of graph prompts. Therefore, the development of a more intuitive graph prompt design with a readable format remains an open problem.

More Downstream Applications Currently, graph prompt learning is primarily applied to node or graph classification tasks on open-source benchmarks [24, 111]. Although potential applications have been discussed in Section 7, the real-world utilization of graph prompt learning remains limited. A notable example is its use in fraud detection within real-world transaction networks, addressing the issue of label scarcity [103]. However, compared to the widespread application of prompting techniques in the NLP domain [1, 71, 122], the potential of graph prompt learning in diverse real-world applications requires further exploration. Overcoming the main challenges of obtaining powerful domain-specific pre-trained graph models and designing suitable prompts for specific application scenarios that exhibit unique characteristics remains crucial.

Transferable Prompt Designs Existing studies on graph prompt learning [55, 82, 21, 13, 12, 20] typically focus on pre-training and prompt tuning using the same dataset, which limits the exploration of more transferable designs and empirical evaluation. Although PRODIGY [35] explores transferability within the same domain and All in One [84] provides empirical results regarding transferability across tasks and domains, the investigation of prompt learning across diverse domains and tasks remains limited. Achieving transferability across diverse tasks and domains requires aligning the task space [84], semantic features [130], and structural patterns [127], which necessitates further theoretical work to provide insights guiding the development of transferable prompt designs.

9.2 Future Directions

With the above analysis on graph prompt, we summarize future directions as follows:

Learning Knowledge from Large Graph Models (LGMs) like LLMs. Currently, graph models are typically tailored to specific domains or tasks, limiting generalization abilities across broader domains or tasks [51, 130]. Therefore, we are expecting the realization of large graph models (LGMs) as a universal tool to be intelligent enough to handle graph tasks across domains [50, 15]. A significant step towards this direction has been taken by Liu et al. [50], who proposed a powerful model (OFA) capable of addressing classification tasks for graphs coming from various domains. However, their approach still relies on LLMs as a general template to distill specific domain knowledge, which we believe can be replaced with suitable graph prompting techniques. Just as textual prompt has been widely used to adapt LLM for diverse applications [71, 100, 2], graph prompting techniques are promising to distill LGMs' knowledge specific for concrete downstream tasks. Applying the graph prompts to LGMs has the potential to revolutionize the field of deep graph learning, leveraging LGMs as a universal tool to tackle different tasks on graphs from diverse domains.

Transferable Learning. As we have discussed in Section 9.1, current graph prompt learning methods [84, 82, 35, 55, 87, 20, 78] are primarily limited to intra-dataset/domain settings, lacking the ability to transfer knowledge across different tasks or domains. While there have been efforts in graph transfer learning [130] to realize domain adaptation, research specifically focused on transferable graph prompting techniques remains limited. To enable the transfer ability, the prompts should be designed to realize the alignment between different task spaces [84], reformulating different tasks on graphs into a uniform template. Besides, both structural [127] and semantics alignment [130] should be realized via suitable graph prompts to enable domain adaptation. It is promising to implement transferable graph prompts, realizing knowledge transfer across domains to extend the generalization and applicability of graph models.

More Theoretical Foundation. Despite the great success of graph prompt learning on various downstream tasks, they mostly draw on the successful experience of prompt tuning on NLP domain [54, 2, 18, 90, 69, 77]. In other words,

most existing graph prompt tuning methods are designed with intuition, and their performance gains are evaluated by empirical experiments. The lack of sufficient theoretical foundations behind the design has led to both performance bottlenecks and poor explainability. Therefore, we believe that building a solid theoretical foundation for graph prompt learning from a graph theory perspective and minimizing the gap between the theoretical foundation and empirical design is also a promising future direction.

More Explainable and Understandable Design. While existing graph prompt learning methods have demonstrated impressive results on various downstream tasks [23, 103, 113], we still lack a clear understanding of what exactly is being learned from the prompts. The black-box learning mode of prompt vectors raises questions about their interpretability and whether we can establish meaningful correspondences between the input data and the prompted graph [84]. These issues are crucial for understanding and interpreting prompts but are currently missing in most graph prompt research. To work towards trustworthy graph prompt learning, it is promising to explore the self-interpretability [8] to enable intuitive explanations of graph prompts. By gaining insights into the learned prompt vectors and structures, we can enhance our understanding of the underlying mechanisms and improve the interpretability of graph prompts. This, in turn, can lead to more effective utilization of prompts for security- or privacy-related downstream tasks.

10 Conclusion

In this survey, we explore the promising intersection between Artificial General Intelligence and graph data by graph prompt. Our unified framework has unveiled a structured understanding of graph prompts, dissecting them into tokens, token structures, and inserting patterns. This framework is a novel contribution, providing clarity and comprehensiveness for researchers and practitioners. By exploring the interplay between graph prompts and models, we've revealed fresh insights into the essence of graph prompts, highlighting their pivotal role in reshaping AI for graph data. With the development of ProG, a Python library, and a dedicated website, we've expanded the graph prompting ecosystem, enhancing collaboration and access to research, benchmark datasets, and code implementations. Our survey outlines a roadmap for the future. The challenges and future directions we've discussed serve as a beacon for the evolving field of graph prompting. With the above work, we hope our survey can push forward a new era of insights and applications in the AGI family.

Acknowledgments

This research is supported in part by project #MMT-p2-23 of the Shun Hing Institute of Advanced Engineering, The Chinese University of Hong Kong, and by grants from the Research Grant Council of the Hong Kong Special Administrative Region, China (No. CUHK 14217622), NSFC Grant No. U1936205. The first author, Dr. Xiangguo Sun, in particular, wants to thank his parents for their kind support during his tough period.

References

[1] A. M. Bran and P. Schwaller, "Transformers and Large Language Models for Chemistry and Drug Discovery," in *Drug Development Supported by Informatics*, 2024.

[2] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal *et al.*, "Language Models Are Few-Shot Learners," in *NeurIPS*, vol. 33, 2020, pp. 1877–1901.

[3] Y. Cao, J. Xu, C. Yang, J. Wang, Y. Zhang, C. Wang, L. Chen, and Y. Yang, "When to pre-train graph neural networks? An answer from data generation perspective!" in *KDD*, 2023.

[4] M. Chen, Z. Liu, C. Liu, J. Li, Q. Mao, and J. Sun, "ULTRA-DP: Unifying Graph Pre-training with Multi-task Graph Dual Prompt," *arXiv preprint*, 2023.

[5] X. Chen, S. Zhang, Y. Xiong, X. Wu, J. Zhang, X. Sun, Y. Zhang, Y. Zhao, and Y. Kang, "Prompt learning on temporal interaction graphs," *arXiv preprint*, 2024.

[6] Z. Chen, H. Mao, H. Li, W. Jin, H. Wen, X. Wei, S. Wang, D. Yin, W. Fan, H. Liu, and J. Tang, "Exploring the Potential of Large Language Models (LLMs) in Learning on Graphs," *ACM SIGKDD Explorations Newsletter*, 2024.

[7] J. Cheng, M. Li, J. Li, and F. Tsung, "Wiener graph deconvolutional network improves graph self-supervised learning," in *AAAI*, 2023, pp. 7131–7139.

[8] E. Dai and S. Wang, "Towards Self-Explainable Graph Neural Network," in *CIKM*, 2021, pp. 302–311.

[9] M. Defferrard, X. Bresson, and P. Vandergheynst, "Convolutional Neural Networks on Graphs with Fast Localized Spectral Filtering," in *NeurIPS*, vol. 29, 2016.

[10] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," *arXiv preprint*, 2019.

[11] C. Edwards, A. Naik, T. Khot, M. Burke, H. Ji, and T. Hope, "SynerGPT: In-Context Learning for Personalized Drug Synergy Prediction and Drug Design," *arXiv preprint*, 2023.

[12] T. Fang, Y. Zhang, Y. Yang, and C. Wang, "Prompt tuning for graph neural networks," *arXiv preprint*, 2022.

[13] T. Fang, Y. Zhang, Y. Yang, C. Wang, and L. Chen, "Universal Prompt Tuning for Graph Neural Networks," in *NeurIPS*, 2023.

[14] X. Fang, L. Liu, J. Lei, D. He, S. Zhang, J. Zhou, F. Wang, H. Wu, and H. Wang, "Geometry-enhanced molecular representation learning for property prediction," *Nature Machine Intelligence*, vol. 4, no. 2, pp. 127–134, 2022.

[15] B. Fatemi, J. Halcrow, and B. Perozzi, "Talk like a graph: Encoding graphs for large language models," *arXiv preprint*, 2023.

[16] C. Finn, P. Abbeel, and S. Levine, "Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks," in *ICML*, 2017.

[17] X. Fu, Z. Wang, Z. Chen, J. Li, Y. Zhu, Z. Lei, C. Shen, Y. Ye, C. Zhang, and J. Li, "Graph Prompting for Graph Learning Models: Recent Advances and Future Directions," *arXiv preprint*, 2025. [Online]. Available: https://arxiv.org/abs/2506.08326v1

[18] T. Gao, A. Fisch, and D. Chen, "Making Pre-Trained Language Models Better Few-Shot Learners," in *ACL*,

2021, pp. 3816–3830.

[19] Z. Gao, X. Sun, Z. Liu, Y. Li, H. Cheng, and J. Li, "Protein Multimer Structure Prediction via Prompt Learning," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=OHpvivXrQr

[20] Q. Ge, Z. Zhao, Y. Liu, A. Cheng, X. Li, S. Wang, and D. Yin, "Enhancing Graph Neural Networks with Structure-Based Prompt," *arXiv preprint*, 2023.

[21] C. Gong, X. Li, J. Yu, C. Yao, J. Tan, C. Yu, and D. Yin, "Prompt Tuning for Multi-View Graph Contrastive Learning," *arXiv preprint*, 2023.

[22] A. Grover and J. Leskovec, "node2vec: Scalable feature learning for networks," in *KDD*, 2016, pp. 855–864.

[23] Y. Guo, C. Yang, Y. Chen, J. Liu, C. Shi, and J. Du, "A Data-Centric Framework to Endow Graph Neural Networks with Out-of-Distribution Detection Ability," in *KDD*, 2023, pp. 638–648.

[24] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive Representation Learning on Large Graphs," in *NeurIPS*, vol. 30, 2017.

[25] B. Hao, C. Yang, L. Guo, J. Yu, and H. Yin, "Motif-Based Prompt Learning for Universal Cross-Domain Recommendation," in *WSDM*, 2024.

[26] A. Hasanzadeh, E. Hajiramezanali, K. Narayanan, N. Duffield, M. Zhou, and X. Qian, "Semi-Implicit Graph Variational Auto-Encoders," in *NeurIPS*, vol. 32, 2019.

[27] K. Hassani and A. H. K. Ahmadi, "Contrastive Multi-View Representation Learning on Graphs," in *ICML*, vol. 119, 2020, pp. 4116–4126.

[28] A. Haviv, J. Berant, and A. Globerson, "BERTese: Learning to Speak to BERT," in *EACL*, 2021, pp. 3618–3623.

[29] M. He, Z. Wei, z. Huang, and H. Xu, "BernNet: Learning Arbitrary Graph Spectral Filters via Bernstein Approximation," in *NeurIPS*, vol. 34, 2021, pp. 14 239–14 251.

[30] Z. Hou, X. Liu, Y. Cen, Y. Dong, H. Yang, C. Wang, and J. Tang, "GraphMAE: Self-Supervised Masked Graph Autoencoders," in *KDD*, 2022, pp. 594–604.

[31] Z. Hou, Y. He, Y. Cen, X. Liu, Y. Dong, E. Kharlamov, and J. Tang, "GraphMAE2: A Decoding-Enhanced Masked Self-Supervised Graph Learner," in *The Web Conference*, 2023, pp. 737–746.

[32] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," *arXiv preprint*, 2021.

[33] W. Hu, B. Liu, J. Gomes, M. Zitnik, P. Liang, V. S. Pande, and J. Leskovec, "Strategies for Pre-training Graph Neural Networks," in *ICLR*, 2020.

[34] Z. Hu, Y. Dong, K. Wang, K.-W. Chang, and Y. Sun, "GPT-GNN: Generative Pre-Training of Graph Neural Networks," in *KDD*, 2020, pp. 1857–1867.

[35] Q. Huang, H. Ren, P. Chen, G. Kržmanc, D. Zeng, P. Liang, and J. Leskovec, "PRODIGY: Enabling In-context Learning Over Graphs," in *NeurIPS*, 2023.

[36] X. Jiang, T. Jia, Y. Fang, C. Shi, Z. Lin, and H. Wang, "Pre-training on Large-Scale Heterogeneous Graph," in *KDD*, 2021, pp. 756–766.

[37] X. Jiang, Y. Lu, Y. Fang, and C. Shi, "Contrastive Pre-Training of GNNs on Heterogeneous Graphs," in *CIKM*, 2021, pp. 803–812.

[38] Z. Jiang, F. F. Xu, J. Araki, and G. Neubig, "How Can We Know What Language Models Know?" *TACL*, vol. 8, pp. 423–438, 2020.

[39] B. Jin, W. Zhang, Y. Zhang, Y. Meng, X. Zhang, Q. Zhu, and J. Han, "Patton: Language Model Pretraining on Text-Rich Networks," in *ACL*, 2023.

[40] M. Jin, Y. Zheng, Y.-F. Li, C. Gong, C. Zhou, and S. Pan, "Multi-Scale Contrastive Siamese Networks for Self-Supervised Graph Representation Learning," in *IJCAI*, 2021, pp. 1477–1483.

[41] W. Jin, T. Derr, Y. Wang, Y. Ma, Z. Liu, and J. Tang, "Node Similarity Preserving Graph Convolutional Networks," in *WSDM*, 2021, pp. 148–156.

[42] D. Kim and A. Oh, "How to Find Your Friendly Neighborhood: Graph Attention Design with Self-Supervision," in *ICLR*, 2021.

[43] B. Lester, R. Al-Rfou, and N. Constant, "The Power of Scale for Parameter-Efficient Prompt Tuning," in *EMNLP*, 2021, pp. 3045–3059.

[44] J. Li, R. Wu, W. Sun, L. Chen, S. Tian, L. Zhu, C. Meng, Z. Zheng, and W. Wang, "What's Behind the Mask: Understanding Masked Graph Modeling for Graph Autoencoders," in *KDD*, 2023, pp. 1268–1279.

[45] S. Li, X. Han, and J. Bai, "AdapterGNN: Efficient Delta Tuning Improves Generalization Ability in Graph Neural Networks," *arXiv preprint*, 2023.

[46] X. L. Li and P. Liang, "Prefix-Tuning: Optimizing Continuous Prompts for Generation," in *ACL-IJCNLP*, 2021, pp. 4582–4597.

[47] X. Li, D. Lian, Z. Lu, J. Bai, Z. Chen, and X. Wang, "GraphAdapter: Tuning Vision-Language Models With Dual Knowledge Graph," in *NeurIPS*, 2023.

[48] Y. Li and B. Hooi, "Prompt-Based Zero-and Few-Shot Node Classification: A Multimodal Approach," *arXiv preprint*, 2023.

[49] Y. Li, Z. Li, P. Wang, J. Li, X. Sun, H. Cheng, and J. X. Yu, "A survey of graph meets large language model: Progress and future directions," *arXiv preprint*, 2023.

[50] H. Liu, J. Feng, L. Kong, N. Liang, D. Tao, Y. Chen, and M. Zhang, "One for All: Towards Training One Graph Model for All Classification Tasks," *arXiv preprint*, 2023.

[51] J. Liu, C. Yang, Z. Lu, J. Chen, Y. Li, M. Zhang, T. Bai, Y. Fang, L. Sun, P. S. Yu, and C. Shi, "Towards Graph Foundation Models: A Survey and Beyond," *arXiv preprint*, 2023.

[52] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, "Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing," *ACM Computing Surveys*, vol. 55, no. 9, pp. 195:1–195:35, 2023.

[53] P. Liu, Y. Ren, and Z. Ren, "GIT-Mol: A Multi-modal Large Language Model for Molecular Science with Graph, Image, and Text," *Computers in Biology and Medicine*, 2024.

[54] X. Liu, K. Ji, Y. Fu, W. Tam, Z. Du, Z. Yang, and J. Tang, "P-Tuning: Prompt Tuning Can Be Comparable to Fine-Tuning Across Scales and Tasks," in *ACL*, vol. 2, 2022, pp. 61–68.

[55] Z. Liu, X. Yu, Y. Fang, and X. Zhang, "Graphprompt:

Unifying Pre-Training and Downstream Tasks for Graph Neural Networks," in *The Web Conference*, 2023, pp. 417–428.

[56] Z. Liu, S. Li, Y. Luo, H. Fei, Y. Cao, K. Kawaguchi, X. Wang, and T.-S. Chua, "MolCA: Molecular Graph-Language Modeling with Cross-Modal Projector and Uni-Modal Adapter," in *EMNLP*, 2023.

[57] Q. Long, Y. Yan, P. Zhang, C. Fang, W. Cui, Z. Ning, M. Xiao, N. Cao, X. Luo, L. Xu, S. Jiang, Z. Fang, C. Chen, X.-S. Hua, and Y. Zhou, "Towards Graph Prompt Learning: A Survey and Beyond," *arXiv preprint*, 2024. [Online]. Available: https://arxiv.org/abs/2408.14520v3

[58] Y. Long, M. Wu, Y. Liu, Y. Fang, C. K. Kwoh, J. Chen, J. Luo, and X. Li, "Pre-training graph neural networks for link prediction in biomedical networks," *Bioinformatics*, vol. 38, no. 8, pp. 2254–2262, 2022.

[59] Y. Ma, N. Yan, J. Li, M. Mortazavi, and N. V. Chawla, "HetGPT: Harnessing the Power of Prompt Tuning in Pre-Trained Heterogeneous Graph Neural Networks," in *Proceedings of the ACM Web Conference 2024*, 2024.

[60] M. Niepert, M. Ahmed, and K. Kutzkov, "Learning Convolutional Neural Networks for Graphs," in *ICML*, 2016, pp. 2014–2023.

[61] M. Ou, P. Cui, J. Pei, Z. Zhang, and W. Zhu, "Asymmetric Transitivity Preserving Graph Embedding," in *KDD*, 2016, pp. 1105–1114.

[62] S. Pan, R. Hu, G. Long, J. Jiang, L. Yao, and C. Zhang, "Adversarially Regularized Graph Autoencoder for Graph Embedding," in *IJCAI*, 2018, pp. 2609–2615.

[63] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, "Unifying Large Language Models and Knowledge Graphs: A Roadmap," *IEEE Transactions on Knowledge and Data Engineering*, 2024.

[64] J. Park, A. Patel, O. Z. Khan, H. J. Kim, and J.-K. Kim, "Graph-Guided Reasoning for Multi-Hop Question Answering in Large Language Models," *arXiv preprint*, 2023.

[65] J. Park, M. Lee, H. J. Chang, K. Lee, and J. Y. Choi, "Symmetric Graph Convolutional Autoencoder for Unsupervised Graph Representation Learning," in *ICCV*, 2019, pp. 6519–6528.

[66] Z. Peng, W. Huang, M. Luo, Q. Zheng, Y. Rong, T. Xu, and J. Huang, "Graph Representation Learning via Graphical Mutual Information Maximization," in *The Web Conference*, 2020, pp. 259–270.

[67] B. Perozzi, R. Al-Rfou, and S. Skiena, "Deepwalk: Online learning of social representations," in *KDD*, 2014, pp. 701–710.

[68] C. Qian, H. Tang, Z. Yang, H. Liang, and Y. Liu, "Can Large Language Models Empower Molecular Property Prediction?" *arXiv preprint*, 2023.

[69] G. Qin and J. Eisner, "Learning How to Ask: Querying LMs with Mixtures of Soft Prompts," in *NAACL-HLT*, 2021, pp. 5203–5212.

[70] J. Qiu, Q. Chen, Y. Dong, J. Zhang, H. Yang, M. Ding, K. Wang, and J. Tang, "GCC: Graph Contrastive Coding for Graph Neural Network Pre-Training," in *KDD*, 2020, pp. 1150–1160.

[71] J. Robinson, C. M. Rytting, and D. Wingate, "Leveraging Large Language Models for Multiple Choice Question Answering," *arXiv preprint*, 2023.

[72] Y. Rong, Y. Bian, T. Xu, W. Xie, Y. WEI, W. Huang, and J. Huang, "Self-Supervised Graph Transformer on Large-Scale Molecular Data," in *NeurIPS*, vol. 33, 2020, pp. 12 559–12 571.

[73] M. T. Rosenstein, Z. Marx, L. P. Kaelbling, and T. G. Dietterich, "To transfer or not to transfer," in *NeurIPS*, vol. 898, 2005.

[74] T. Schick and H. Schütze, "Few-Shot Text Generation with Natural Language Instructions," in *EMNLP*, 2021, pp. 390–402.

[75] ——, "It's Not Just Size That Matters: Small Language Models Are Also Few-Shot Learners," in *NAACL-HLT*, 2021, pp. 2339–2352.

[76] Y. Shi, Z. Huang, S. Feng, H. Zhong, W. Wang, and Y. Sun, "Masked Label Prediction: Unified Message Passing Model for Semi-Supervised Classification," in *IJCAI*, vol. 2, 2021, pp. 1548–1554.

[77] T. Shin, Y. Razeghi, R. L. L. IV, E. Wallace, and S. Singh, "Autoprompt: Eliciting Knowledge from Language Models with Automatically Generated Prompts," in *EMNLP*, 2020, pp. 4222–4235.

[78] R. Shirkavand and H. Huang, "Deep Prompt Tuning for Graph Transformers," *arXiv preprint*, 2023.

[79] A. Subramonian, "MOTIF-Driven Contrastive Learning of Graph Representations," in *AAAI*, vol. 35, 2021, pp. 15 980–15 981.

[80] F.-Y. Sun, J. Hoffmann, V. Verma, and J. Tang, "InfoGraph: Unsupervised and Semi-supervised Graph-Level Representation Learning via Mutual Information Maximization," in *ICLR*, 2020.

[81] M. Sun, J. Xing, H. Wang, B. Chen, and J. Zhou, "MoCL: Data-driven Molecular Fingerprint via Knowledge-aware Contrastive Learning from Molecular Graph," in *KDD*, 2021, pp. 3585–3594.

[82] M. Sun, K. Zhou, X. He, Y. Wang, and X. Wang, "GPPT: Graph Pre-Training and Prompt Tuning to Generalize Graph Neural Networks," in *KDD*, 2022, pp. 1717–1727.

[83] X. Sun, H. Yin, B. Liu, H. Chen, J. Cao, Y. Shao, and N. Q. Viet Hung, "Heterogeneous Hypergraph Embedding for Graph Classification," in *WSDM*, 2021, pp. 725–733.

[84] X. Sun, H. Cheng, J. Li, B. Liu, and J. Guan, "All in One: Multi-Task Prompting for Graph Neural Networks," in *KDD*, 2023, pp. 2120–2131.

[85] S. Suresh, P. Li, C. Hao, and J. Neville, "Adversarial Graph Augmentation to Improve Graph Contrastive Learning," in *NeurIPS*, vol. 34, 2021, pp. 15 920–15 933.

[86] Q. Tan, N. Liu, X. Huang, S.-H. Choi, L. Li, R. Chen, and X. Hu, "S2GAE: Self-Supervised Graph Autoencoders are Generalizable Learners with Graph Masking," in *Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining*, ser. WSDM '23, 2023, pp. 787–795. [Online]. Available: https://doi.org/10.1145/3539597.3570404

[87] Z. Tan, R. Guo, K. Ding, and H. Liu, "Virtual Node Tuning for Few-shot Node Classification," in *KDD*, 2023, pp. 2177–2188.

[88] S. Thakoor, C. Tallec, M. G. Azar, R. Munos, P. Veličković, and M. Valko, "Bootstrapped representa-

tion learning on graphs," in *ICLR*, 2021.

[89] Y. Tian, H. Song, Z. Wang, H. Wang, Z. Hu, F. Wang, N. V. Chawla, and P. Xu, "Graph Neural Prompting with Large Language Models," *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.

[90] M. Tsimpoukelli, J. L. Menick, S. Cabi, S. M. A. Eslami, O. Vinyals, and F. Hill, "Multimodal Few-Shot Learning with Frozen Language Models," in *NeurIPS*, vol. 34, 2021, pp. 200–212.

[91] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph Attention Networks," in *ICLR*, 2018.

[92] P. Velickovic, W. Fedus, W. L. Hamilton, P. Liò, Y. Bengio, and R. D. Hjelm, "Deep Graph Infomax," in *ICLR*, 2019.

[93] C. Wang, S. Pan, G. Long, X. Zhu, and J. Jiang, "MGAE: Marginalized Graph Autoencoder for Graph Clustering," in *CIKM*, 2017, pp. 889–898.

[94] H. Wang, T. Fu, Y. Du, W. Gao, K. Huang, Z. Liu *et al.*, "Scientific discovery in the age of artificial intelligence," *Nature*, vol. 620, no. 7972, pp. 47–60, 2023.

[95] J. Wang, D. Chen, C. Luo, X. Dai, L. Yuan, Z. Wu, and Y.-G. Jiang, "ChatVideo: A Tracklet-centric Multimodal and Versatile Video Understanding System," *arXiv preprint*, 2023.

[96] P. Wang, K. Agarwal, C. Ham, S. Choudhury, and C. K. Reddy, "Self-Supervised Learning of Contextual Embeddings for Link Prediction in Heterogeneous Networks," in *The Web Conference*, 2021, pp. 2946–2957.

[97] R. Wang, Y. Li, S. Lin, W. Wu, H. Xie, Y. Xu, and J. C. Lui, "Common neighbors matter: fast random walk sampling with common neighbor awareness," *TKDE*, vol. 35, no. 5, pp. 4570–4584, 2022.

[98] X. Wang, Y. Zhang, and C. Shi, "Hyperbolic heterogeneous information network embedding," in *AAAI*, vol. 33, 2019, pp. 5337–5344.

[99] X. Wang, N. Liu, H. Han, and C. Shi, "Self-supervised Heterogeneous Graph Neural Network with Co-contrastive Learning," in *KDD*, 2021, pp. 1726–1736.

[100] Y. Wang, N. Lipka, Ryan A. Rossi, Alexa Siu, Ruiyi Zhang, and Tyler Derr, "Knowledge Graph Prompting for Multi-Document Question Answering," *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.

[101] Z. Wen and Y. Fang, "Augmenting Low-Resource Text Classification with Graph-Grounded Pre-Training and Prompting," in *SIGIR*, 2023, pp. 506–516.

[102] ——, "Prompt Tuning on Graph-augmented Low-resource Text Classification," *IEEE Transactions on Knowledge and Data Engineering*, 2024.

[103] Z. Wen, Y. Fang, Y. Liu, Y. Guo, and S. Hao, "Voucher Abuse Detection with Prompt-based Fine-tuning on Graph Neural Networks," in *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*, 2023.

[104] J. Wu, X. Wang, F. Feng, X. He, L. Chen, J. Lian, and X. Xie, "Self-supervised Graph Learning for Recommendation," in *SIGIR*, 2021.

[105] J. Wu, S. Li, A. Deng, M. Xiong, and H. Bryan, "Prompt-and-Align: Prompt-Based Social Alignment for Few-Shot Fake News Detection," in *CIKM*, 2023.

[106] X. Wu, K. Zhou, M. Sun, X. Wang, and N. Liu, "A Survey of Graph Prompting Methods: Techniques, Applications, and Challenges," *arXiv preprint*, 2023.

[107] Y. Wu, R. Xie, Y. Zhu, F. Zhuang, X. Zhang, L. Lin, and Q. He, "Personalized Prompt for Sequential Recommendation," *IEEE Transactions on Knowledge and Data Engineering*, 2024.

[108] Y. Xie, Z. Xu, and S. Ji, "Self-Supervised Representation Learning via Latent Graph Prediction," in *ICML*, 2022, pp. 24 460–24 477.

[109] Y. Xie, Z. Xu, J. Zhang, Z. Wang, and S. Ji, "Self-Supervised Learning of Graph Neural Networks: A Unified Review," *TPAML*, vol. 45, no. 2, pp. 2412–2429, 2023.

[110] D. Xu, W. Cheng, D. Luo, H. Chen, and X. Zhang, "InfoGCL: Information-Aware Graph Contrastive Learning," in *NeurIPS*, vol. 34, 2021, pp. 30 414–30 425.

[111] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How Powerful are Graph Neural Networks?" in *ICLR*, 2018.

[112] C. Yang, D. Bo, J. Liu, Y. Peng, B. Chen, H. Dai *et al.*, "Data-centric graph learning: A survey," *IEEE Transactions on Big Data*, 2025.

[113] H. Yang, X. Zhao, Y. Li, H. Chen, and G. Xu, "An Empirical Study Towards Prompt-Tuning for Graph Contrastive Pre-Training in Recommendations," in *NeurIPS*, 2023.

[114] Z. Yi, I. Ounis, and C. Macdonald, "Contrastive Graph Prompt-tuning for Cross-domain Recommendation," *TOIS*, 2023.

[115] Y. You, T. Chen, Y. Sui, T. Chen, Z. Wang, and Y. Shen, "Graph contrastive learning with augmentations," in *NeurIPS*, 2020, pp. 5812–5823.

[116] J. Yu, H. Yin, X. Xia, T. Chen, L. Cui, and N. Q. V. Hung, "Are Graph Augmentations Necessary? Simple Graph Contrastive Learning for Recommendation," in *SIGIR*, 2022.

[117] J. Yu, H. Yin, X. Xia, T. Chen, J. Li, and Z. Huang, "Self-Supervised Learning for Recommender Systems: A Survey," *TKDE*, pp. 1–20, 2023.

[118] H. Zhang, X. Li, and L. Bing, "Video-LLaMA: An Instruction-tuned Audio-Visual Language Model for Video Understanding," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2023.

[119] J. Zhang, H. Zhang, C. Xia, and L. Sun, "Graph-Bert: Only Attention is Needed for Learning Graph Representations," *arXiv preprint*, 2020.

[120] J. Zhang, Y. Dong, Y. Wang, J. Tang, and M. Ding, "ProNE: Fast and Scalable Network Representation Learning," in *IJCAI*, 2019, pp. 4278–4284.

[121] M. Zhang and Y. Chen, "Link Prediction Based on Graph Neural Networks," in *NeurIPS*, 2018.

[122] T. Zhang, F. Ladhak, E. Durmus, P. Liang, K. McKeown, and T. B. Hashimoto, "Benchmarking Large Language Models for News Summarization," *Transactions of the Association for Computational Linguistics*, 2024.

[123] W. Zhang, Y. Zhu, M. Chen, Y. Geng, Y. Huang, Y. Xu, W. Song, and H. Chen, "Structure Pretraining and Prompt Tuning for Knowledge Graph Transfer," in *The Web Conference*, 2023, pp. 2581–2590.

[124] Z. Zhang, H. Li, Z. Zhang, Y. Qin, X. Wang, and W. Zhu, "Large graph models: A perspective," *arXiv preprint*,

2023.

[125] H. Zhao, A. Chen, X. Sun, H. Cheng, and J. Li, "All in one and one for all: A simple yet effective method towards cross-domain graph pretraining," in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2024.

[126] H. Zhao, S. Liu, C. Ma, H. Xu, J. Fu, Z.-H. Deng, L. Kong, and Q. Liu, "GIMLET: A Unified Graph-Text Model for Instruction-Based Molecule Zero-Shot Learning," in *Advances in Neural Information Processing Systems 36*, 2023.

[127] W. Zhao, Q. Wu, C. Yang, and J. Yan, "GraphGLOW: Universal and Generalizable Structure Learning for Graph Neural Networks," in *KDD*, 2023, pp. 3525–3536.

[128] Y. Zhu, Y. Xu, F. Yu, Q. Liu, S. Wu, and L. Wang, "Graph Contrastive Learning with Adaptive Augmentation," in *The Web Conference*, 2021, pp. 2069–2080.

[129] Y. Zhu, J. Guo, and S. Tang, "SGL-PT: A Strong Graph Learner with Graph Prompt Tuning," *arXiv preprint*, 2023.

[130] Y. Zhu, Y. Wang, H. Shi, Z. Zhang, and S. Tang, "Graph-Control: Adding Conditional Control to Universal Graph Pre-trained Models for Graph Domain Transfer Learning," in *Proceedings of the ACM Web Conference 2024*, 2024.